UNIT 3

1) **Justify the impact of normalization on database? Explain 1st, 2nd normal form, 3rd normal form and BCNF with example.**

💠 Impact (Importance) of Normalization on Database

Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity.

It divides large, complex tables into smaller, related ones and defines relationships among them.

✅ Impact / Advantages:

1. Removes Data Redundancy:
   Duplicate data is eliminated; storage is optimized.
   → e.g., Student details stored once instead of repeating in every record.
2. Ensures Data Consistency:
   Updates made in one place reflect everywhere.
3. Improves Data Integrity:
   Reduces chances of anomalies (insertion, deletion, update).
4. Enhances Query Performance:
   Smaller, well-structured tables are easier to maintain and search.
5. Simplifies Maintenance:
   Easier to modify structure or add new data without affecting existing data.
6. Avoids Anomalies:
   - Insertion anomaly – can't insert due to missing data.
   - Deletion anomaly – deleting one record may delete useful data.
   - Update anomaly – updating one place but not others causes inconsistency.

🧩 Normal Forms

Normalization is carried out in steps called **Normal Forms (NF)**.

1️⃣ First Normal Form (1NF)

Rule:

- Each cell must contain **a single atomic value** (no repeating groups or arrays).
- Each record must be unique.

🔹 Example (Before 1NF):

StudentID	StudentName	Subjects
1	Ravi	DBMS, SE
2	Priya	CN, DBMS, AI

Problem: "Subjects" column has multiple values — not atomic.

✅ After 1NF:

StudentID	StudentName	Subject
1	Ravi	DBMS
1	Ravi	SE
2	Priya	CN
2	Priya	DBMS
2	Priya	AI

🧩 Normal Forms

Normalization is carried out in steps called **Normal Forms (NF)**.

1️⃣ First Normal Form (1NF)

Rule:

- Each cell must contain **a single atomic value** (no repeating groups or arrays).
- Each record must be unique.

🔹 Example (Before 1NF):

StudentID	StudentName	Subjects
1	Ravi	DBMS, SE
2	Priya	CN, DBMS, AI

Problem: "Subjects" column has multiple values — not atomic.

✅ After 1NF:

StudentID	StudentName	Subject
1	Ravi	DBMS
1	Ravi	SE
2	Priya	CN
2	Priya	DBMS
2	Priya	AI

2️⃣ Second Normal Form (2NF)

Rules:

- Must be in **1NF**, and
- **No partial dependency:**
  No non-key attribute should depend on a **part of a composite primary key**.

🔹 **Example (Before 2NF):**

StudentID	CourseID	StudentName	CourseName
1	DB01	Ravi	DBMS
2	CN01	Priya	CN

Composite Key: (StudentID, CourseID)

Problem:

- StudentName depends **only on StudentID**
- CourseName depends **only on CourseID**
  → Partial dependencies exist.

✅ **After 2NF:**

Split into two tables:

Student Table

StudentID	StudentName
1	Ravi
2	Priya

Course Table

CourseID	CourseName
DB01	DBMS
CN01	CN

Enrollment Table

StudentID	CourseID
1	DB01
2	CN01

Now no attribute depends on part of the key → Table is in **2NF**.

3️⃣ Third Normal Form (3NF)

Rules:

- Must be in **2NF**, and
- **No transitive dependency:**
  Non-key attributes should not depend on other non-key attributes.

🔹 **Example (Before 3NF):**

StudentID	StudentName	DeptID	DeptName
1	Ravi	D1	Computer
2	Priya	D2	IT

Problem:

- DeptName depends on DeptID (non-key → non-key dependency).
  → Transitive dependency exists.

✅ **After 3NF:**

Student Table

StudentID	StudentName	DeptID
1	Ravi	D1
2	Priya	D2

Department Table

DeptID	DeptName
D1	Computer
D2	IT

Now, every non-key attribute depends **only on the primary key** → Table is in **3NF**.

4️⃣ Boyce-Codd Normal Form (BCNF)

Rule:

- For every **functional dependency (X → Y),**
  X must be a super key.
- A stronger version of 3NF.

🔹 **Example (Before BCNF):**

Course	Instructor	Room
DBMS	Smith	R1
OS	John	R2
DBMS	John	R1

Functional dependencies:

- (Course → Room)
- (Room → Instructor)

Here, **Room → Instructor** violates BCNF because "Room" is not a super key.

✅ **After BCNF:**

Table 1: Room–Instructor

Room	Instructor
R1	Smith
R2	John

Table 2: Course–Room

Course	Room
DBMS	R1
OS	R2

Now all determinants are super keys → Table is in **BCNF**.

📜 Summary Table

Normal Form	Removes	Key Rule
1NF	Repeating groups	Atomic values
2NF	Partial dependency	No attribute depends on part of composite key
3NF	Transitive dependency	Non-key attribute depends only on key
BCNF	Remaining anomalies	Every determinant is a super key

2) Elaborate the significance of codd's rule. Explain 12 rules proposed by codd's.

💠 Significance of Codd's Rules

E. F. Codd, the father of the Relational Database Model, proposed 12 rules (actually 13, numbered 0 to 12) in 1985 to define what qualifies as a true relational database management system (RDBMS).

✅ Significance:

1. They define the foundation of relational databases.
2. Help to differentiate relational databases from simple file-based systems.
3. Ensure data independence, integrity, and consistency.
4. Encourage standardization in database design and operations.
5. Provide a guideline for database vendors to develop robust RDBMS software.
6. Improve data security, logical structure, and ease of access.

🧩 Codd's 12 Rules (Rule 0–12)

Let's understand each in simple terms 👇

Rule No.	Rule Name	Explanation
Rule 0:	Foundation Rule	Any system that claims to be an RDBMS must manage data **only through relational capabilities** — i.e., data must be stored in tables, not files or other formats.
Rule 1:	Information Rule	All information (data, metadata) in the database must be **represented in table form**, with rows and columns.
Rule 2:	Guaranteed Access Rule	Each piece of data (value) should be **accessible using table name, primary key, and column name** — no need for physical addresses or pointers.
Rule 3:	Systematic Treatment of Null Values	The system must support **NULL values** (unknown or inapplicable data) in a consistent way, distinct from zero or empty string.
Rule 4:	Dynamic Online Catalog Based on the Relational Model	The database **catalog (data dictionary)** should also be stored in the form of tables and accessible using SQL.
Rule 5:	Comprehensive Data Sublanguage Rule	The database must support at least one **comprehensive language** (like SQL) for data definition, manipulation, and control (DDL, DML, DCL).

Rule 6:	View Updating Rule	All theoretically updatable **views** (virtual tables) must be updatable through the system.
Rule 7:	High-Level Insert, Update, and Delete Rule	The system must support **set-level operations** (on multiple rows) for insertion, update, and deletion—not just one record at a time.
Rule 8:	Physical Data Independence	Changes in **storage structure or access methods** should not affect the logical structure or user operations.
Rule 9:	Logical Data Independence	Changes in **logical structure** (like adding new fields or tables) should not affect application programs.
Rule 10:	Integrity Independence	Integrity constraints (like primary key, foreign key) must be **defined in the database**, not in the application program.
Rule 11:	Distribution Independence	Users should be **unaware of data distribution** — whether it is stored locally or on remote servers (transparent access).
Rule 12:	Nonsubversion Rule	If a system provides **low-level access (bypassing SQL)**, it must not bypass integrity rules or security constraints.

3) What is the impact of insert, update and delete anomaly on overall design of database? How is normalization used to remove these anomalies?

💠 Impact of Insert, Update, and Delete Anomalies on Database Design

In a **poorly designed (unnormalized)** database, data is often **repeated** and **stored redundantly**.
This leads to several problems known as **data anomalies**, which affect **data integrity, accuracy, and consistency**.

🔹 1️⃣ Insertion Anomaly

Meaning:
Occurs when certain data **cannot be inserted** into the database **without the presence of other data**.

Example:

StudentID	StudentName	Course
1	Ravi	DBMS

If a new course (e.g., "AI") starts but no student has enrolled yet,
we **cannot insert** that course because the table requires student information.

Impact:

- Leads to **incomplete data** storage.
- Prevents addition of valid new entries.

🔹 2️⃣ Update Anomaly

Meaning:

Occurs when the same data is **stored in multiple places**, and a change in one place is **not reflected** in others.

Example:

StudentID	StudentName	Course	Instructor
1	Ravi	DBMS	Smith
2	Priya	DBMS	Smith

If instructor "Smith" changes to "John," we must update **both rows**.
If we forget to update one, data becomes inconsistent.

Impact:

- Causes **inconsistency** in data.
- Leads to **errors** and loss of reliability.

🔹 3️⃣ Deletion Anomaly

Meaning:

Occurs when deleting some data **accidentally removes useful information**.

Example:

StudentID	StudentName	Course
1	Ravi	DBMS

If we delete Ravi's record (student drops out),
we also **lose information about the course "DBMS"**, even though it still exists.

Impact:

- Results in **unintentional data loss.**
- Affects **data integrity** and **future use**.

💠 How Normalization Removes These Anomalies

Normalization is the process of organizing data into related tables to **eliminate redundancy** and **avoid anomalies**.

✅ 1NF (First Normal Form)

- Removes repeating groups and ensures atomic values.
- Prevents **insertion anomalies** due to non-atomic fields.

✅ 2NF (Second Normal Form)

- Removes **partial dependency** (attributes depending on part of composite key).
- Ensures data about entities like students, courses, and instructors are stored in **separate tables**.
- Reduces **update anomalies**.

✅ 3NF (Third Normal Form)

- Removes **transitive dependencies** (non-key → non-key).
- Keeps data **logically separated**, avoiding **deletion anomalies**.

✅ BCNF (Boyce–Codd Normal Form)

- Strengthens dependency rules so that **every determinant is a key**, ensuring **maximum consistency**.

📑 Summary Table

Type of Anomaly	Cause	Effect on Design	Normalization Solution
Insertion	Missing data dependency	Incomplete data	1NF & 2NF
Update	Data redundancy	Inconsistent data	2NF & 3NF
Deletion	Data coupling	Loss of useful info	3NF & BCNF

🔹 Conclusion

Unnormalized tables cause serious **data integrity issues** due to insert, update, and delete anomalies.
By applying **normalization (up to 3NF or BCNF)**, a database becomes **consistent, reliable, and efficient**, improving the **overall design and data management**.

4) Enlist differences between 1st, 2nd, 3rd and, BCNF

💠 Differences between 1NF, 2NF, 3NF, and BCNF

Normal Form	Main Rule / Condition	Removes	Dependency Type Allowed	Example Problem Fixed	Key Point
1NF (First Normal Form)	Each cell should contain **atomic (single) values** and each record should be unique.	Repeating groups, multivalued attributes	Only simple key dependency	Multiple values in one column (e.g., Subjects = DBMS, SE)	Ensures atomicity
2NF (Second Normal Form)	Must be in 1NF and **no partial dependency** (non-key attribute should depend on full composite key).	Partial dependency	Fully functional dependency	Attributes depend only on part of composite key	Ensures full dependency
3NF (Third Normal Form)	Must be in 2NF and **no transitive dependency** (non-key → non-key).	Transitive dependency	Direct key → non-key only	DeptName depends on DeptID, not StudentID	Ensures data depends only on key
BCNF (Boyce-Codd Normal Form)	Must be in 3NF and for every dependency **X → Y**, X must be a **super key**.	Remaining anomalies from 3NF	Only super key dependencies	Room → Instructor type dependency	Stronger version of 3NF

📝 Summary:

- 1NF → Removes repeating data
- 2NF → Removes partial dependency
- 3NF → Removes transitive dependency
- BCNF → Removes remaining anomalies by ensuring every determinant is a key

5) Explain different features of good relational database design.

💠 Features of a Good Relational Database Design

A **good relational database design** ensures that data is stored efficiently, accurately, and can be easily retrieved and maintained.

It should minimize redundancy and maintain data integrity while supporting the required operations.

✅ 1️⃣ Data Integrity and Consistency

- Ensures that data in the database remains **accurate and reliable**.
- Uses **Primary Keys**, **Foreign Keys**, and **Constraints** (like `NOT NULL`, `UNIQUE`, `CHECK`) to maintain correctness.

  Example: Preventing duplicate student IDs in a student table.

✅ 2️⃣ Data Redundancy Control

- Avoids storing the same data multiple times.
- Reduces chances of **update, insert, and delete anomalies**.
- Achieved through **Normalization** (1NF, 2NF, 3NF, BCNF).

✅ 3️⃣ Proper Use of Keys

- **Primary Key** → uniquely identifies each record.
- **Foreign Key** → maintains relationships between tables.
- **Candidate Key** and **Alternate Key** → provide flexibility and uniqueness.

  These ensure logical connections between tables.

✅ 4️⃣ Normalization

- Organizes data into well-structured tables.
- Eliminates redundancy and ensures **data dependency is logical**.
- Improves **data integrity** and simplifies maintenance.

✅ 5️⃣ Referential Integrity

- Maintains valid relationships between tables using **foreign keys**.
- Ensures that references between tables remain consistent (no orphan records).

  Example: A student record cannot refer to a non-existing department.

✅ 6️⃣ Data Independence

- Changes in **physical storage** or **logical structure** should not affect application programs.
- Ensures **flexibility and scalability** of the system.

✅ 7️⃣ Simplicity and Clarity

- Tables should represent **one entity** (e.g., Student, Course, Department).
- Column names should be **meaningful and self-explanatory**.
- Simplifies database understanding and reduces design complexity.

✅ 8️⃣ Efficient Query Performance

- Proper **indexing**, **foreign key relationships**, and **optimized table structures** help in faster data retrieval.
- Queries should run efficiently even on large datasets.

✅ 9️⃣ Scalability and Flexibility

- The design should allow for **future growth** — new tables, fields, or relationships can be added without major restructuring.

✅ 🔟 Security and Access Control

- Supports **user authentication**, **role-based permissions**, and **access levels** to protect sensitive data.

6) Explain Codd's rules with suitable examples

💠 Codd's 12 Rules of Relational Database (with Examples)

E.F. Codd proposed **13 rules (Rule 0 to 12)** to define what a **true Relational Database Management System (RDBMS)** should be.

Each rule ensures **data integrity, independence, and relational consistency**.

Rule 0 – Foundation Rule

> A system must use relational capabilities to manage the database.

Meaning:

If a system calls itself an RDBMS, it must **store and manipulate data using relational tables** only.

Example:

MySQL or Oracle uses **tables (rows and columns)** — not flat files or hierarchies.

Rule 1 – Information Rule

> All information in a database must be represented logically as values in **tables**.

Example:

Student data is stored in a table:

StudentID	Name	Age
1	Ravi	20

Everything — even metadata — is represented in **rows and columns**.

Rule 2 – Guaranteed Access Rule

> Each data item must be accessible by **table name, primary key, and column name**.

Example:

To access Ravi's name:

→ Table: `Student`, Key: `StudentID = 1`, Column: `Name`.

No need to know physical storage location.

Rule 3 – Systematic Treatment of Null Values

> Null values must be **supported and treated consistently**.

Example:

If a student's phone number is unknown → store as `NULL`, not 0 or "N/A".

This helps differentiate **unknown** vs **empty** data.

Rule 4 – Dynamic Online Catalog Based on the Relational Model

> The database catalog (data dictionary) must also be stored in **table form** and accessible using SQL.

Example:

You can query metadata using:

```sql
SELECT * FROM INFORMATION_SCHEMA.TABLES;
```

This retrieves all tables in the database.

Rule 5 – Comprehensive Data Sublanguage Rule

> The database must support at least **one language (like SQL)** that can perform all operations:

- Data Definition (DDL)
- Data Manipulation (DML)
- Data Control (DCL)
- Transaction Control (TCL)

Example:

SQL provides commands like:

`CREATE TABLE`, `SELECT`, `INSERT`, `UPDATE`, `DELETE`, `COMMIT`.

Rule 6 – View Updating Rule

> All views (virtual tables) that can theoretically be updated must be **updatable** through the system.

Example:

If you create a view:

```sql
CREATE VIEW StudentNames AS SELECT Name FROM Student;
```

Then updating this view should update the original table as well.

Rule 7 – High-Level Insert, Update, and Delete

> The system must support **set-level operations** (not just one record at a time).

Example:
You can update multiple records in one query:

```sql
UPDATE Student SET Age = Age + 1 WHERE Dept = 'CSE';
```

→ Updates all CSE students at once.

Rule 8 – Physical Data Independence

> Changes in **storage structure or access methods** must not affect the logical data model.

Example:
Moving the database to another disk or changing the indexing structure should not affect how users query data.

Rule 9 – Logical Data Independence

> Changes in **logical structure** (like adding new columns) should not affect application programs.

Example:
If a new column `Email` is added to the `Student` table,
existing queries using `Name` and `Age` should still work.

Rule 10 – Integrity Independence

> Integrity constraints (like primary key, foreign key, etc.) should be **defined in the database**, not in application code.

Example:

```sql
CREATE TABLE Student (
  StudentID INT PRIMARY KEY,
  Name VARCHAR(50),
  DeptID INT,
  FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);
```

Integrity rules are stored in the database itself.

Rule 11 – Distribution Independence

> A user should be unaware of whether data is stored **locally or distributed** across multiple locations.

Example:

A query like:

```sql
SELECT * FROM Student;
```

Should give the same result whether data is on one server or multiple remote servers.

Rule 12 – Nonsubversion Rule

> If a system provides low-level (record-level) access, it must not bypass integrity or security rules defined by the RDBMS.

Example:

Even if an administrator edits a data file directly,

they cannot violate constraints like primary keys or foreign keys.

7) Explain entity and referential integrity constraints used in SQL.

💠 Entity and Referential Integrity Constraints in SQL

Integrity constraints are rules that ensure the accuracy and consistency of data in a database.
Two important integrity constraints in the Relational Model are:

- Entity Integrity Constraint
- Referential Integrity Constraint

🔹 1️⃣ Entity Integrity Constraint

Definition:

Entity Integrity ensures that every table (relation) has a unique and identifiable row — that is, Primary Key values must be unique and not NULL.

Explanation:

- Each table must have a Primary Key that uniquely identifies every record.
- The Primary Key column(s) cannot contain NULL values, because NULL represents unknown data, and an unknown value cannot uniquely identify a record.
- This maintains the uniqueness and existence of each entity (row).

Example:

sql Copy code

```sql
CREATE TABLE Student (
  StudentID INT PRIMARY KEY,
  Name VARCHAR(50),
  Age INT
);
```

StudentID	Name	Age
1	Ravi	20
2	Priya	21
NULL	Aman	19 ❌ *Not allowed*
1	Rohan	22 ❌ *Duplicate key not allowed*

✅ Here, StudentID uniquely identifies each student — enforcing Entity Integrity.

🔷 2 Referential Integrity Constraint

Definition:

Referential Integrity ensures that **relationships between tables remain consistent.**
A **Foreign Key** in one table must always refer to a **valid Primary Key** in another table.

Explanation:

- A **Foreign Key** establishes a relationship between two tables.
- The foreign key value must either:
  - Match an existing primary key value in the parent table, or
  - Be NULL (if allowed).
- This prevents **orphan records** — records referring to non-existing entities.

Example:

sql Copy code

```sql
CREATE TABLE Department (
  DeptID INT PRIMARY KEY,
  DeptName VARCHAR(50)
);

CREATE TABLE Student (
  StudentID INT PRIMARY KEY,
  Name VARCHAR(50),
  DeptID INT,
  FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);
```

Department Table

DeptID	DeptName
1	Computer
2	IT

Student Table

StudentID	Name
1	Ravi
2	Priya
3	Rohan

✅ Referential Integrity ensures that every student's **DeptID** exists in the **Department** table.

📜 Summary Table

Constraint	Ensures	Key Used	NULL Allowed?	Example Violation
Entity Integrity	Each record is uniquely identified	Primary Key	❌ Not allowed	Duplicate or NULL key
Referential Integrity	Valid relationship between tables	Foreign Key	✅ Allowed (if optional)	Foreign key refers to non-existing primary key

💬 Conclusion

- **Entity Integrity** maintains the **uniqueness** of records within a table.
- **Referential Integrity** maintains the **consistency of relationships** between tables.

Together, they ensure the **reliability, accuracy, and stability** of relational databases.

8) **What are relational integrity constraints. Explain with example Domain constraints, Referential-Integrity and enterprise constraints.**

💠 Relational Integrity Constraints

Definition:

Relational Integrity Constraints are **rules** that ensure the **accuracy, consistency, and validity** of data in a relational database.

These constraints prevent invalid data from being entered into tables and maintain relationships among data.

🔹 1. Domain Constraints

Definition:

Domain constraints ensure that the **values in each column (attribute)** come from a **specific domain** — i.e., they must be of a valid data type, range, or set of values.

Example:

sql Copy code

```sql
CREATE TABLE Student (
   RollNo INT,
   Name VARCHAR(50),
   Age INT CHECK (Age >= 18 AND Age <= 25)
);
```

✅ Here,

- `Age` must be between **18 and 25** → domain constraint.
- If you try to insert `Age = 30`, the database will reject it.

🔹 2. Entity Integrity Constraint

Definition:

Entity integrity ensures that every **table (relation)** has a **primary key**, and the **primary key cannot be NULL or duplicate**.

This guarantees that each record can be **uniquely identified**.

Example:

```sql
CREATE TABLE Student (
   RollNo INT PRIMARY KEY,
   Name VARCHAR(50),
   Age INT
);
```

✅ Here,

- `RollNo` is the **primary key**.
- You cannot insert a record with `NULL` or duplicate `RollNo`.

🔷 3. Referential Integrity Constraint

Definition:

Referential integrity ensures that **foreign key values** in one table must **match** a **primary key value** in another table — or be **NULL**.

It maintains **valid relationships** between tables.

Example:

```sql
CREATE TABLE Department (
   DeptID INT PRIMARY KEY,
   DeptName VARCHAR(50)
);

CREATE TABLE Employee (
   EmpID INT PRIMARY KEY,
   EmpName VARCHAR(50),
   DeptID INT,
   FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);
```

✅ Here,

- Each `DeptID` in the **Employee** table must exist in the **Department** table.
- If a department is deleted, you can't have employees assigned to that deleted department (unless handled using `ON DELETE` rules).

🔷 4. Enterprise Constraints (Business Rules)

Definition:

Enterprise constraints are **user-defined rules** that enforce specific **business policies or logic** in the organization.

They go beyond standard SQL constraints.

Example:

- A company policy states that **salary cannot exceed ₹1,00,000**.
- Or a student can register for a **maximum of 5 subjects**.

```sql
CREATE TABLE Employee (
   EmpID INT PRIMARY KEY,
   EmpName VARCHAR(50),
   Salary DECIMAL(10,2) CHECK (Salary <= 100000)
);
```

✅ Here,

The `CHECK` condition enforces the enterprise rule on salary.

📑 Summary Table

Constraint Type	Purpose	Example
Domain Constraint	Validates data type & range	`Age BETWEEN 18 AND 25`
Entity Integrity	Ensures unique, non-null primary key	`PRIMARY KEY (RollNo)`
Referential Integrity	Maintains valid foreign key relationships	`FOREIGN KEY (DeptID)`
Enterprise Constraint	Enforces business rules	`CHECK (Salary <= 100000)`

9) What is functional dependency? Explain its use in database design. [9] Consider the following schema. Student (RollNo, Branch_code, Marks_Obtained, Exam_Name, Total_Marks) Identify the functional dependencies and check whether the given schema is in 3NF or not. If not justify and convert the schema into 3NF.

1. What is a Functional Dependency (FD)?

A **functional dependency** `X → Y` between attribute sets in a relation R means: **for any two tuples in R, if they agree on X, they must agree on Y.**
In other words, the value of **X functionally determines** the value of Y.

Example: `RollNo → Branch_code` means a given `RollNo` always has exactly one `Branch_code`.

---

2. Uses of Functional Dependencies in database design

1. **Identify candidate keys** — FDs help find minimal attribute sets that determine all other attributes.
2. **Normalize relations** — FDs reveal redundancies (partial & transitive dependencies) and guide decomposition into 1NF, 2NF, 3NF, BCNF.
3. **Prevent anomalies** — by decomposing on FDs you reduce insert/update/delete anomalies.
4. **Enforce integrity & constraints** — FDs correspond to keys and constraints implemented in SQL (PRIMARY KEY, UNIQUE, FOREIGN KEY, CHECK).
5. **Query optimization & schema design** — knowing FDs enables better indexing and query planning.

3. Given schema

`Student(RollNo, Branch_code, Marks_Obtained, Exam_Name, Total_Marks)`

a) Identify likely functional dependencies

From typical semantics we can infer:

1. `RollNo → Branch_code`
   (a student has one branch)
2. `(RollNo, Exam_Name) → Marks_Obtained`
   (the marks a student obtained in a particular exam)
3. `Exam_Name → Total_Marks`
   (each exam has a fixed total marks)
4. From (2) and (1)/(3) we can derive other dependencies but the above are the main ones.

b) Candidate key(s)

A minimal key that identifies tuples in this relation is the composite:

- **Key = (RollNo, Exam_Name)**
  because a student can have multiple exams and an exam has many students; the pair uniquely identifies a record of marks.

c) Check whether the schema is in 3NF

Recall: to be in **3NF** relation must be in **2NF** and have **no transitive dependencies** of a non-prime attribute on the key (i.e., every non-prime attribute must be either dependent on a key, a superkey, or be part of a candidate key).

- **Non-prime attributes** (attributes not part of any candidate key): `Branch_code`, `Marks_Obtained`, `Total_Marks`.
- `Marks_Obtained` is fully dependent on the key `(RollNo, Exam_Name)` → OK.
- **Problem (partial dependencies):**
  - `RollNo → Branch_code` — here `Branch_code` depends on **part** of the composite key (`RollNo`), not on the whole key. This is a **partial dependency** → violates **2NF** (hence also violates 3NF).
  - `Exam_Name → Total_Marks` — `Total_Marks` depends on **part** of the composite key (`Exam_Name`) → another **partial dependency** → violates **2NF**.
- There is no transitive dependency of the form `Key → A → B` with A non-key and B non-key (except that the partial dependencies could be seen as transitive if you treat part-key → non-key), but since partial dependencies already break 2NF, the schema is **not in 3NF**.

Conclusion: The given schema is **not in 3NF**.

4. Convert the schema into 3NF (decomposition)

Decompose to eliminate partial dependencies while preserving data and enabling lossless join and dependency preservation.

Natural decomposition based on FDs:

1. **Student** table (store student attributes)

   ```sql
   Student(RollNo PRIMARY KEY, Branch_code)
   ```

   FD preserved: `RollNo → Branch_code`.
2. **Exam** table (store exam attributes)

   ```sql
   Exam(Exam_Name PRIMARY KEY, Total_Marks)
   ```

   FD preserved: `Exam_Name → Total_Marks`.

3. **Result** (or Marks) table (relationship between student and exam)

```sql
Result(RollNo, Exam_Name, Marks_Obtained,
       PRIMARY KEY (RollNo, Exam_Name),
       FOREIGN KEY (RollNo) REFERENCES Student(RollNo),
       FOREIGN KEY (Exam_Name) REFERENCES Exam(Exam_Name))
```

FD preserved: `(RollNo, Exam_Name) → Marks_Obtained`.

Why this decomposition is correct

- **Removes partial dependencies:** `Branch_code` moved to `Student` (depends only on `RollNo`), `Total_Marks` moved to `Exam` (depends only on `Exam_Name`). `Marks_Obtained` remains in `Result` and depends on whole composite key.
- **Lossless-join:**
  - `Student ⋈ Result` on `RollNo` is lossless because `RollNo` is key of `Student`.
  - `Exam ⋈ Result` on `Exam_Name` is lossless because `Exam_Name` is key of `Exam`.
    Thus joining `Student`, `Exam`, and `Result` reconstructs the original relation without spurious tuples.
- **Dependency preservation:** The three FDs listed are preserved in the decomposed relations.
- **Resulting relations are in 3NF:**
  - In `Student`, key `RollNo` uniquely determines `Branch_code`. No non-key depends on non-key.
  - In `Exam`, key `Exam_Name` uniquely determines `Total_Marks`.
  - In `Result`, key `(RollNo, Exam_Name)` uniquely determines `Marks_Obtained`.

5. Final answer (summary)

- **Functional dependencies identified:**
  `RollNo → Branch_code`
  `(RollNo, Exam_Name) → Marks_Obtained`
  `Exam_Name → Total_Marks`
- **Original schema status:** Not in 3NF (violates 2NF due to partial dependencies).
- **3NF decomposition:**
  `Student(RollNo PK, Branch_code)`
  `Exam(Exam_Name PK, Total_Marks)`
  `Result(RollNo, Exam_Name, Marks_Obtained)` with `PK(RollNo,Exam_Name)` and FKs to `Student` and `Exam`.

10) What is decomposition? Consider the relation F (FN, PN, C, D) with the following Functional Dependencies: FD1: FN, PN ->C FD2: C ->D FD3: D -> F CEGP013091 49.248.216.238 16/05/2024 09:33:14 static-238 If F is decomposed in to F1 (FN,PN,C) and F2 (C,D). check decomposition is lossless or lossy?

🧩 1. What is a Functional Dependency (FD)?

👉 Think of it like **cause and effect** between columns (attributes) in a table.

If one column's value **decides** another column's value,
we say that column **functionally determines** the other.

Example:

RollNo	Name	Branch
1	Riya	CS
2	Aman	IT
3	Riya	CS

Here, each **RollNo** has only one **Name** and one **Branch**.
So we can say:

- `RollNo → Name`
- `RollNo → Branch`

That's a **Functional Dependency**.

It means:

> If RollNo is same → Name & Branch will also be same.

3) Decomposition — plain meaning

- **Decompose relation** `R` **into** `R1, R2, ...` means replace a big table with smaller tables holding subsets of attributes.
- **Goal:** remove redundancy and anomalies while keeping the ability to reconstruct original data (by joins).

Two desired properties:

1. **Lossless-join:** join of decomposed tables returns exactly the original tuples (no spurious rows).
2. **Dependency-preserving:** all original FDs can be enforced by constraints on the new tables (without having to join).

🧱 2. What is Decomposition?

Decomposition simply means **breaking one big table into smaller tables** to remove repetition and make data cleaner.

You do this without losing any information.

🧮 Example

Let's say we have one big table called **Student_Result**:

RollNo	Name	Subject	Marks
1	Riya	Math	80
1	Riya	Science	90
2	Aman	Math	85
2	Aman	Science	88

Here, the **student name** is repeating again and again.

So we can **decompose (split)** it into two smaller tables:

1️⃣ Student Table

RollNo	Name
1	Riya
2	Aman

2️⃣ Marks Table

RollNo	Subject	Marks
1	Math	80
1	Science	90
2	Math	85
2	Science	88

Now, there's no repetition of names.

✅ We can **join both tables** (by RollNo) to get back the original big table again.
That means — **Decomposition is lossless**.

🚫 3. What is Lossless and Lossy Decomposition?

Type	Meaning	Example
Lossless	When you split a table and can join them back without losing or changing data	Splitting Student_Result into Student and Marks (we got back same data using RollNo)
Lossy	When you split and after joining back you get **extra or missing rows**	If we split wrongly — for example, remove RollNo and try to join only by Name — then two students with same name will mix up (wrong data)

🧩 The question in short:

We have one big table:

`R(FN, PN, C, D)`

We split (decompose) it into two smaller tables:

- `R1(FN, PN, C)`
- `R2(C, D)`

We want to check:

👉 Is this decomposition **lossless** or **lossy**?

Step 2️⃣: The rule to check

To check if decomposition is **lossless**, we look at the **common column(s)** between the two tables.

Here:

Common column = `{C}`

Now we check —

does this common column **decide all other columns** of at least one table?

That means:

👉 Does **C → (R1 − R2)** or **C → (R2 − R1)** hold true?

Step 3: What is R1 − R2 and R2 − R1?

Let's find out the attributes that are **not common**:

- `R1 - R2` = attributes in R1 but **not** in R2
  → `{FN, PN}`
- `R2 - R1` = attributes in R2 but **not** in R1
  → `{D}`

Step 4: Now check if common column C decides one of these.

We're given FDs:

- FD1: (FN, PN) → C
- FD2: C → D

So we can see:

✅ `C → D` (that's FD2)

That means **C determines D**.
And D is in `R2 - R1`.

So, **C → (R2 − R1)** holds true.

Step 5: What does this mean?

It means:

> The common column (C) can **uniquely determine** everything in one of the decomposed tables (R2).

Hence, when we join R1 and R2 using C, we'll get **the same original table** — no wrong combinations or missing info.

✅ **So the decomposition is lossless.**

🧠 In short:

Step	Explanation	Result
Common column	{C}	—
Check rule	Does C → (R1 − R2) or C → (R2 − R1)?	—
Find differences	R1−R2 = {FN, PN}, R2−R1 = {D}	—
Given FD	C → D	—
So	C → (R2−R1) holds	✅
Therefore	Decomposition is **lossless**	🎉

UNIT 4

11) Suppose a transaction Ti issues a read command on data item Q.How time-stamp based protocol decides whether to allow the operation to be executed or not using time-stamp based protocol of concurrency control. Explain in detail time stamp based protocol.

1 Introduction

In a multi-user database system, **Concurrency Control** is used to ensure correct results when multiple transactions execute simultaneously.

The **Timestamp Based Protocol** is a **non-locking** method of concurrency control which orders all transactions according to their **timestamps**.

---

2 Concept of Timestamp

- Each transaction **Ti** is assigned a unique **timestamp TS(Ti)** when it starts.
- Timestamp indicates the **order of arrival** or **age** of the transaction.
- If **TS(Ti) < TS(Tj)** → Transaction **Ti** is older than **Tj**.

---

3 Information Stored for Each Data Item (Q)

For every data item **Q**, two timestamp values are maintained:

Symbol	Meaning
read_TS(Q)	The largest timestamp of any transaction that successfully read Q.
write_TS(Q)	The largest timestamp of any transaction that successfully wrote Q.

Initially, both are set to **0**.

4️⃣ Rules of Timestamp Ordering Protocol

A) When Transaction Ti issues `READ(Q)`

Let `t = TS(Ti)`

- If `t < write_TS(Q)`
  → A **younger transaction** has already written Q.
  → Reading this value will violate the timestamp order.
  ✅ **Action: Abort Ti** and restart with new timestamp.
- Else
  → Allow the read operation.
  → Update `read_TS(Q) = max(read_TS(Q), t)`.

B) When Transaction Ti issues `WRITE(Q)`

Let `t = TS(Ti)`

- If `t < read_TS(Q)`
  → Some younger transaction has already read Q.
  → Writing now will make older Ti overwrite a value that a newer transaction has read.
  ✅ **Action: Abort Ti.**
- Else if `t < write_TS(Q)`
  → A younger transaction has already written Q.
  → Writing now would be out of order.
  ✅ **Action:** Either **abort Ti** or **ignore write** (Thomas's Write Rule).
- Else
  → Perform the write.
  → Update `write_TS(Q) = t`.

5️⃣ Thomas's Write Rule (Optimization)

If a transaction tries to write an old value (its timestamp is smaller than the latest write), the write can be **ignored instead of aborting** the transaction.

👉 Example:
If `write_TS(Q)=50` and `TS(Ti)=40`, then the write of Ti is obsolete.
Instead of aborting, just **ignore** it.

6 Example

Transaction	Operation	Condition Checked	Action	Updated Values
T1 (TS=5)	READ(Q)	5 ≥ write_TS(Q)=0	Allowed	read_TS(Q)=5
T2 (TS=10)	WRITE(Q)	10 ≥ read_TS(Q)=5	Allowed	write_TS(Q)=10
T1 (TS=5)	READ(Q)	5 < write_TS(Q)=10	Not Allowed	T1 Aborted

✅ Hence, protocol ensures serial order = T1 before T2.

---

7 Advantages

1. **No Deadlocks** (no locks used).
2. Ensures **serializability** automatically.
3. **Thomas's Rule** reduces unnecessary aborts.

---

8 Disadvantages

1. May cause **many aborts** if older transactions are delayed.
2. Needs **unique timestamp generation**.
3. Can suffer from **starvation** of long transactions.

9 Conclusion

Timestamp based concurrency control ensures that **transactions execute in timestamp order** and maintain **serializability** without using locks.
It prevents conflicts by **aborting or ignoring** operations that violate the order.

✅ **In short:**

> When Ti issues a read on Q, it is allowed **only if** `TS(Ti) ≥ write_TS(Q)`.
> Otherwise, Ti is aborted.
> This maintains a conflict-free serial order based on timestamps.

12) Explain the concept of conflict serializability with suitable example. Since every conflict-serializable schedule is view serializable, why do we emphasize conflict serializability rather than view serializability?

1 Introduction

In a multi-user database, when two or more transactions execute concurrently, their operations can interleave. To ensure correctness, the result of concurrent execution must be the same as some serial execution (one after another).
This property is called Serializability — and it ensures consistency of the database.

There are two main types:

1. Conflict Serializability
2. View Serializability

---

2 Conflict Serializability – Concept

A schedule (sequence of read/write operations) is said to be conflict serializable if it can be transformed into a serial schedule by swapping non-conflicting operations.

---

3 Definition of Conflicting Operations

Two operations are said to be in conflict if:

1. They belong to different transactions, and
2. They operate on the same data item, and
3. At least one of them is a WRITE operation.

4 Types of Conflicts

Operation Type	Conflict Example
Read–Write	`T1: read(Q)` and `T2: write(Q)`
Write–Read	`T1: write(Q)` and `T2: read(Q)`
Write–Write	`T1: write(Q)` and `T2: write(Q)`

5 Example of Conflict Serializability

Consider two transactions:

T1:

```scss
read(A)
write(A)
```

T2:

```scss
read(A)
write(A)
```

Schedule S1:

```css
T1: read(A)
T2: read(A)
T1: write(A)
T2: write(A)
```

Let's find conflicts:

Operation Pair	Conflict Type	Order
T1.write(A) – T2.read(A)	Write–Read	T1 before T2
T1.write(A) – T2.write(A)	Write–Write	T1 before T2

👉 All conflicts show T1 → T2 ordering.

Hence, schedule is **conflict equivalent** to serial schedule T1 → T2,

✅ So S1 is **Conflict Serializable**.

6 Conflict Graph (Precedence Graph)

To test conflict serializability:

1. Create a **node** for each transaction.
2. Draw a **directed edge** Ti → Tj if an operation of Ti conflicts with an operation of Tj and Ti's operation occurs **before** Tj's.
3. If the graph has **no cycle**, the schedule is **conflict serializable**.

---

7 Example of Non-Serializable Schedule

If we get both `T1 → T2` and `T2 → T1` edges,
→ a cycle occurs → **Not serializable.**

8 View Serializability

A schedule is **view serializable** if it produces the **same final results** and **reads-from relations** as some serial schedule.
It is **broader** than conflict serializability — some view-serializable schedules are not conflict-serializable.

---

9 Why Emphasize Conflict Serializability?

Reason	Explanation
✅ **Easier to test**	Conflict serializability can be checked using a simple **precedence graph** (acyclic test).
⚙️ **Efficient**	Algorithm is straightforward and computationally feasible.
📊 **View serializability is complex**	Checking view serializability is **NP-Complete**, hence not practical for large systems.
🛠️ **Conflict implies view**	Every **conflict-serializable** schedule is also **view-serializable**, so it's sufficient to ensure conflict serializability.

🔟 Conclusion

- **Conflict Serializability** ensures correctness by controlling the order of conflicting operations.
- It guarantees **consistency** and is **easier to implement** using precedence graph.
- Since every conflict-serializable schedule is also view-serializable,
  → **DBMSs prefer conflict serializability** for practical and efficient concurrency control.

✅ In short:

> A schedule is **conflict-serializable** if it can be transformed into a serial schedule by swapping non-conflicting operations.
> We emphasize conflict serializability because it is **easier and faster to check** than view serializability.

13) State and explain the ACID properties. During its execution a transaction passes though several states, until it finally commits or aborts. List all possible sequences of states through which a transaction may pass. Explain the situations when each state transition occurs.

🧩 ACID Properties of a Transaction

Each transaction in a database must satisfy the **ACID** properties to ensure **data reliability and consistency**.

Property	Full Form	Explanation	Example
A – Atomicity	All-or-nothing	A transaction should complete fully or not at all.	If ₹500 is transferred from A → B, both debit and credit must happen; if one fails, both are undone.
C – Consistency	Data integrity maintained	Database must remain consistent before and after the transaction.	Account balances must follow rules (no negative total).
I – Isolation	Transactions independent	Transactions should not interfere with each other.	Two users withdrawing money from same account won't affect each other's balance.
D – Durability	Permanent results	Once committed, changes must survive even if system crashes.	After transfer completes, data is permanently stored in disk.

🧠 Transaction States in DBMS

A transaction moves through several states during its life cycle.
Let's explain each state in the order shown in your diagram 👇

1️⃣ Active State

- The transaction is currently executing.
- It performs **read/write operations** on data items.
- Example: While transferring money, both accounts are being updated.

➡️ **Next:**

- Goes to **Partially Committed** if all steps executed successfully.
- Goes to **Failed** if any error or crash occurs.

2️⃣ Partially Committed State

- The transaction has completed its final statement.
- Temporary changes are made in memory but **not yet saved permanently**.

➡️ **Next:**

- If system is fine → goes to **Committed State**.
- If crash or error → goes to **Failed State**.

3️⃣ Committed State

- All changes are **permanently saved** in the database.
- The transaction is successful and durable.

➡️ **Next:**

- Goes to **Terminated State** (finished successfully).

4️⃣ Failed State

- Transaction **cannot continue** due to an error, constraint violation, or system crash.

➡️ **Next:**

- System performs **Rollback** → moves to **Aborted State**.

5 Aborted State

- All temporary changes are undone (rolled back).
- Database returns to its previous consistent state.

➡ Next:

- System may restart the transaction or terminate it permanently → Terminated State.

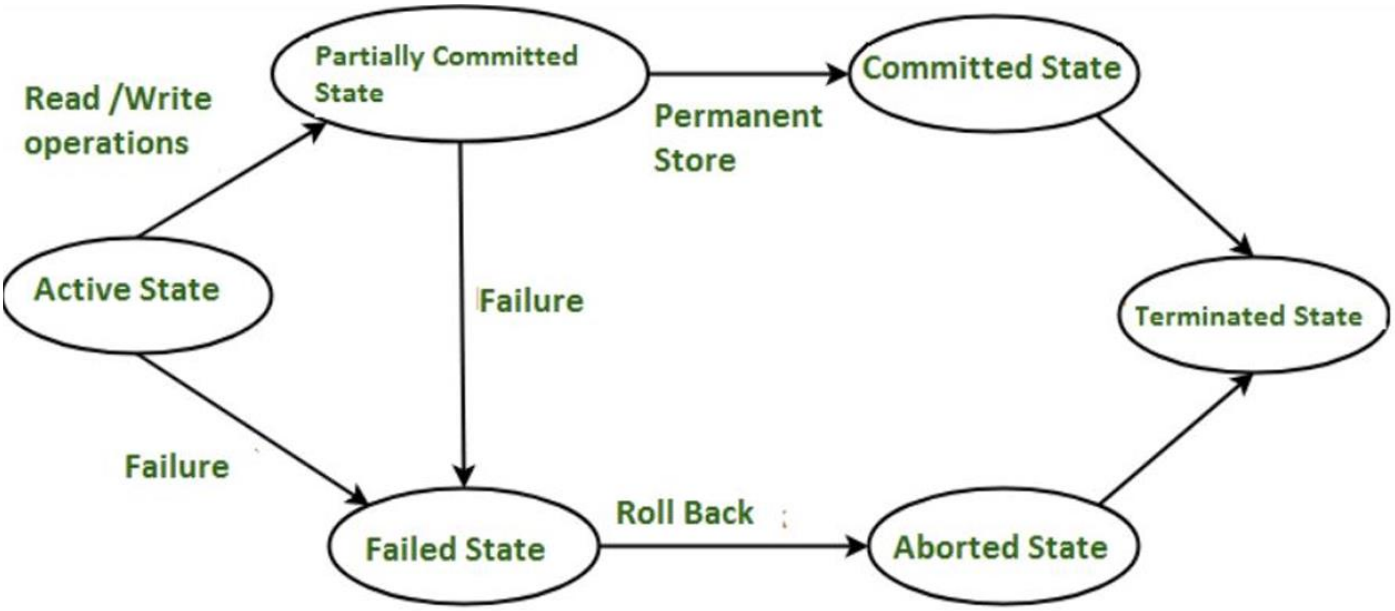
---

6 Terminated State

- The transaction has completed execution — either successfully committed or aborted.
- It is now removed from the transaction table.

🔄 Possible Sequences of State Transitions

Sequence	Meaning
Active → Partially Committed → Committed → Terminated	Successful transaction ✅
Active → Failed → Aborted → Terminated	Failed transaction ❌
Active → Partially Committed → Failed → Aborted → Terminated	Failure occurred just before commit ❌



Transaction States in DBMS

14) A transaction may be waiting for more time for an Exclusive (X) lock on an item, while a sequence of other transactions request and are granted as Shared (S) lock on the same item. What is this problem? How is it solved by two phase lock protocol?

🧠 Problem Description

When a **transaction wants an Exclusive (X) lock** on a data item (to update or write it),
but **other transactions keep taking Shared (S) locks** on that same item,
the waiting transaction can **never get the X lock** — because S locks keep coming one after another.

⚠️ This situation is called Starvation (or Indefinite Postponement)

💡 Example

Let's say we have one data item **A**, and transactions:

Transaction	Operation
T1	Wants to **write(A)** → needs **X lock**
T2	Wants to **read(A)** → needs **S lock**
T3	Wants to **read(A)** → needs **S lock**
T4	Wants to **read(A)** → needs **S lock**

🔍 What happens:

1. T1 requests an **X lock** on A — but currently, T2 already has an **S lock**.
   → So T1 has to **wait**.
2. Before T2 releases it, T3 also requests **S lock** — and gets it (since multiple S locks are allowed).
3. Then T4 requests **S lock**, and also gets it.
4. T1 keeps waiting forever for an X lock, because **there's always some S lock active**.

💥 **This is Starvation Problem.**

🧩 Why does it happen?

Because **X lock requests have lower priority** —
every time a new transaction requests a **Shared (S) lock**, it gets it immediately,
keeping the **Exclusive (X) lock request waiting** indefinitely.

🧰 Solution: Two-Phase Locking Protocol (2PL)

The **Two-Phase Locking (2PL)** protocol ensures **serializability** and helps **avoid starvation**.

🔶 Two-Phase Locking Protocol Rules

Every transaction has **two phases**:

1. **Growing Phase**
   - Transaction **can acquire locks** (S or X).
   - **Cannot release** any lock.
2. **Shrinking Phase**
   - Transaction **can release locks**,
   - **Cannot acquire** any new lock.

💡 How 2PL Helps

- Once a transaction releases **any lock**, it cannot obtain more locks → prevents others from entering conflicting phases indefinitely.
- By ensuring that all locks are obtained first (growing phase), transactions execute in a **conflict-serializable** manner.
- **Waiting transactions** (like one requesting an X lock) will eventually get a chance once current S locks are released — preventing starvation.

✅ Example with 2PL

1. T2, T3, T4 read A (Shared locks).
2. After finishing reads, they **release** their S locks (shrinking phase).
3. Now, T1 can **acquire the X lock** and update A (growing phase → shrinking phase).

So, the waiting transaction **eventually proceeds** — no indefinite waiting.

📑 Summary Table

Concept	Description
Problem Name	Starvation / Indefinite Postponement
Cause	Continuous granting of S locks while X lock waits
Effect	Write transaction keeps waiting forever
Solution	Two-Phase Locking Protocol (2PL)
How Solved	Ensures lock ordering, serializability, and fair execution

---

🏁 In short:

- When a transaction keeps waiting for X lock while others keep getting S locks → it's **Starvation**.
- **Two-Phase Locking (2PL)** protocol prevents this by controlling when locks can be acquired and released, ensuring fair and serializable execution.

15) Write a short note on : i) Log based recovery ii) Shadow Paging

15) i) Log-Based Recovery

👉 Concept:

Log-based recovery is a **technique to recover data** in a database after a **system crash or failure**.
The system keeps a **log file** (or journal) that records **all operations** performed by transactions.

💾 What is a Log?

A **log** is a sequence of records that stores all the changes made by transactions.
Each log record contains:

```pgsql
<Transaction ID, Data Item, Old Value, New Value>
```

💡 Example:

If transaction T1 updates item A from 100 to 200
→ The log record will be:
`<T1, A, 100, 200>`

📍Types of Log Records:

1. **<T, start>** – Transaction T has started.
2. **<T, X, old_value, new_value>** – T modified data item X.
3. **<T, commit>** – Transaction T has completed successfully.
4. **<T, abort>** – Transaction T has been rolled back.

🛠️ Recovery Process:

1. **Undo** – If a transaction **did not commit**, all its changes are **reversed** using the log's old values.
2. **Redo** – If a transaction **committed**, all its operations are **redone** using the new values in the log.

✅ Advantage:

- Ensures **Atomicity** and **Durability** in database.
- No committed data is lost even after a crash.

ii) Shadow Paging

👉 **Concept:**

Shadow paging is another **recovery mechanism** that avoids logging.

It uses **two copies (pages)** of data — one is the **shadow page** (original data), and the other is the **current page** (modified data).

📁 **Structure:**

- A **page table** stores the addresses of all data pages.
- When a transaction starts, the system creates a **shadow copy** of this page table.

💡 **Working:**

1. The **shadow page table** always points to **stable (old)** data.
2. The **current page table** is used for modifications during the transaction.
3. If the transaction **commits**, the system **replaces the shadow table** with the current one.
4. If the transaction **fails**, the **shadow table** is kept, and changes are discarded.

🧪 Example:

Suppose a page contains value 100.

- Shadow page: 100
- Current page (after update): 200

  If transaction commits → shadow updated to 200.

  If abort → shadow (100) remains valid.

✅ Advantages:

- No need for undo/redo operations.
- Faster recovery after crash.

❌ Disadvantages:

- Copying large page tables is expensive.
- Not suitable for large databases.

📘 Summary Table:

Feature	Log-Based Recovery	Shadow Paging
Method	Uses log records	Uses shadow copies
Undo/Redo	Required	Not required
Overhead	Less storage overhead	High copying overhead
Suitable For	Large DB systems	Small to medium DB

✅ In short:

- **Log-based recovery** → keeps track of every change (undo/redo possible).
- **Shadow paging** → keeps two copies (safe revert possible).

16) Explain the two-phase lock protocol for concurrency control. Also explain its two versions: strict two-phase lock protocol and rigorous two-phase lock protocol.

Two-Phase Locking (2PL) Protocol

👉 Concept:

The **Two-Phase Locking (2PL) protocol** is a **concurrency control technique** used in databases to ensure that **transactions are executed in a serializable way** (no conflicts or data inconsistency).

It works by **dividing the transaction into two phases**:

1. Growing Phase
2. Shrinking Phase

🔹 1. Growing Phase:

- A transaction **can acquire locks** (Shared or Exclusive).
- But it **cannot release any lock** during this phase.
- The phase continues until the transaction acquires all the locks it needs.
- The point where it acquires its **last lock** is called the **Lock Point**.

🔹 2. Shrinking Phase:

- A transaction **can release locks**,
- But it **cannot acquire any new lock** after releasing one.

📘 Example:

Let's say Transaction **T1** needs data items **A** and **B**.

Step	Operation	Phase
T1 acquires lock on A	Growing	
T1 acquires lock on B	Growing	
T1 releases lock on A	Shrinking	
T1 releases lock on B	Shrinking	

Once it starts releasing locks → it **cannot take new locks**.
This ensures **serializability** (safe ordering of transactions).

✅ **Advantage:**

- Guarantees **conflict-serializable** schedules.
- Prevents inconsistent data updates.

❌ **Disadvantage:**

- May lead to **deadlocks**, as transactions wait for each other's locks.

Types of Two-Phase Locking Protocols

1️⃣ Strict Two-Phase Locking Protocol

Rule:

- A transaction **holds all Exclusive (X) locks** until it **commits or aborts**.
- Shared (S) locks can be released earlier.

📘 **Example:**

If T1 updates data item A, it keeps **X(A)** locked until commit →

This prevents other transactions from reading uncommitted data (**no dirty reads**).

✅ **Advantage:**

- Simplifies recovery (no need to undo uncommitted updates).
- Avoids **cascading rollbacks**.

2️⃣ Rigorous Two-Phase Locking Protocol

Rule:

- A transaction **holds both Shared (S)** and **Exclusive (X)** locks **until it commits or aborts**.

So **no lock is released** until the transaction ends.

✅ **Advantage:**

- The **safest** version of 2PL.
- Guarantees **serializability** and **strictness** (no dirty read or write).
- Easier to recover from failures.

❌ **Disadvantage:**

- May cause **more waiting** and **lower concurrency**.

📊 Comparison Table

Type	Lock Release	Advantages	Drawbacks
Basic 2PL	After lock point	Ensures serializability	Can cause deadlock
Strict 2PL	Release X-locks at commit	Avoids cascading rollback	Slightly less concurrency
Rigorous 2PL	Release all locks at commit	Highest safety, easy recovery	Least concurrency

📘 Summary:

- **2PL** ensures **conflict serializability**.
- **Strict 2PL** avoids **cascading rollbacks**.
- **Rigorous 2PL** ensures **maximum safety and recoverability**.

17) What is R-timestamp(Q) and W-timestamp(Q) Explain the necessary condition used by time stamp ordering protocol to execute for a read / write operation.

Time-Stamp Based Concurrency Control

The **Time-Stamp Ordering (TSO) protocol** is a **concurrency control method** that ensures **serializability** by ordering transactions according to their **timestamps**.

Each transaction is assigned a **unique timestamp (TS)** when it starts.

This timestamp decides the **order in which the transaction should appear in a serial schedule**.

🔹 Key Terms

1. **TS(T)** → Timestamp of transaction **T**
   It indicates when the transaction **started**.
2. **R-timestamp(Q)** → **Read Timestamp** of data item **Q**
   ➤ It is the **largest timestamp** among all transactions that have **successfully read** data item **Q**.
3. **W-timestamp(Q)** → **Write Timestamp** of data item **Q**
   ➤ It is the **largest timestamp** among all transactions that have **successfully written** data item **Q**.

🔹 Basic Idea

When a transaction **Ti** issues a **read(Q)** or **write(Q)** operation,
the protocol compares **TS(Ti)** with **R-timestamp(Q)** and **W-timestamp(Q)**
to decide whether the operation is **allowed**, **ignored**, or **rolled back**.

🔶 Rules for Time-Stamp Ordering Protocol

1️⃣ Read Operation — `read(Q)`

When transaction **Ti** tries to read data item **Q**:

- **Case 1:**
  If **TS(Ti) < W-timestamp(Q)**
  → It means **Ti** is *older* than the last writer of Q.
  → **Ti** wants to read a value that has been overwritten by a *younger* transaction.
  → ❌ **Operation rejected**, and **Ti is rolled back**.
- **Case 2:**
  If **TS(Ti) ≥ W-timestamp(Q)**
  → The read is **allowed**.
  → Update: **R-timestamp(Q) = max(R-timestamp(Q), TS(Ti))**

2️⃣ Write Operation — `write(Q)`

When transaction **Ti** tries to write data item **Q**:

- **Case 1:**

  If **TS(Ti) < R-timestamp(Q)**

  → It means a **younger transaction** has already read the old value of Q.

  → **Ti's write** would create inconsistency.

  → ❌ **Reject** and **roll back** Ti.

- **Case 2:**

  If **TS(Ti) < W-timestamp(Q)**

  → A **younger transaction** has already written Q.

  → So Ti's write is **outdated**.

  → ❌ **Reject and roll back** Ti.

- **Case 3:**

  If both above cases are false

  → ✅ **Allow the write**, and set

  W-timestamp(Q) = TS(Ti)

🔹 Example

Transaction	Timestamp	Operation	Condition	Result
T1	5	write(Q)	TS(T1)=5, W-timestamp(Q)=0	Allowed → W-timestamp(Q)=5
T2	10	read(Q)	TS(T2)=10 ≥ W-timestamp(Q)=5	Allowed → R-timestamp(Q)=10
T3	7	write(Q)	TS(T3)=7 < R-timestamp(Q)=10	Rejected (Rollback T3)

🔹 Advantages

- Ensures **conflict-serializable schedules**.
- No **deadlocks** (because no waiting for locks).

🔹 Disadvantages

- Frequent **transaction rollbacks** (if timestamps conflict).
- May reduce system throughput.

18) To ensure atomicity despite failures we use Recovery Methods Explain in detail following Log-Based Recovery methods with example. i) Deferred Database Modifications ii) Immediate Database Modifications

Log-Based Recovery

🔹 What is Recovery?

Recovery means restoring the **database to a consistent state** after a **failure** (like system crash, power failure, etc.).

To ensure **Atomicity** (A in ACID), the DBMS uses **log files** that record all operations of a transaction.

🔹 What is a Log?

A **log** is a **sequential record** of all transaction activities.
Each entry in the log contains information like:

pgsql Copy code

```pgsql
< Transaction ID, Data Item, Old Value, New Value >
```

Example:

Copy code

```
<T1, X, 100, 150>
```

Means: Transaction **T1** changed data item **X** from **100** → **150**.

Types of Log-Based Recovery Methods

---

i) Deferred Database Modification (Deferred Update)

◆ Concept:

- All updates of a transaction are **not written** to the database **immediately**.
- They are **stored temporarily in the log** until the transaction **commits**.
- Only **after COMMIT**, the actual changes are made to the database.

◆ Steps:

1. When a transaction starts, write a **<T_i start>** log record.
2. For each update:

   Record **<T_i, X, new_value>** in the log, but **do not update the database** yet.
3. When the transaction commits:
   - Write **<T_i commit>** in the log.
   - Apply all updates to the database.

◆ Recovery Actions:

- If **system fails before commit** → **Ignore** the updates (since they were not applied).
- If **system fails after commit** → **Redo** the updates using the log.

◆ Example:

Step	Action	Log Entry	Database
1	T1 starts	`<T1 start>`	—
2	T1 updates A=100→200	`<T1, A, 200>`	A=100
3	T1 commits	`<T1 commit>`	Then update A=200

If failure occurs:

- Before commit → **No change needed (nothing written)**
- After commit → **Redo A=200**

◆ Advantages:

- No need to **UNDO** anything (since uncommitted data never written).
- Ensures **atomicity** easily.

◆ Disadvantages:

- Slower, since updates are delayed until commit.

ii) Immediate Database Modification (Immediate Update)

◆ Concept:

- Updates are **applied to the database immediately** (before commit).
- But all updates are **also logged** (both old and new values).

◆ Steps:

1. When transaction starts → write `<T_i start>` in log.
2. For every update → write `<T_i, X, old_value, new_value>` in log.
3. Apply update **immediately** to the database.
4. When transaction commits → write `<T_i commit>`.

◆ Recovery Actions:

- If **system fails before commit** → **Undo** (rollback) using **old values** in log.
- If **system fails after commit** → **Redo** using **new values** in log.

◆ Example:

Step	Action	Log Entry	Database
1	T1 starts	`<T1 start>`	—
2	T1 updates A=100→200	`<T1, A, 100, 200>`	A=200
3	System crashes before commit	—	A=200

During recovery:

- Since no commit → **Undo** using old value → A=100 restored.

If crash happens **after commit**, then **Redo** (A=200).

🔶 Advantages:

- Faster, because database updates can happen in real-time.
- Good for systems needing high throughput.

🔶 Disadvantages:

- Both **Undo** and **Redo** may be required.
- Slightly more complex than deferred update.

🧩 Comparison Table

Feature	Deferred Update	Immediate Update
Update applied	After commit	Before commit
Log content	New values only	Old and new values
Recovery action	Only **Redo**	**Undo + Redo**
Data safety	Safer (no uncommitted data)	May contain uncommitted data
Speed	Slower	Faster

✅ **In summary:**

- **Deferred Update:** "Write changes only after commit." → Needs **Redo only**
- **Immediate Update:** "Write immediately but log everything." → Needs **Undo + Redo**

19) What is recoverable schedule? Why is recoverability of schedule desirable?Are there any circumstance under Which it could be desirable to allow non recoverable schedular? Explain your answer.

🔷 Recoverable Schedule

Definition:

A **schedule** is said to be **recoverable** if a transaction **commits only after all the transactions from which it has read data have also committed**.

👉 In simple words —
If one transaction **uses (reads)** data that was **written by another**,
it should **wait for that other transaction to commit** before it commits itself.

This ensures that if the first transaction fails or rolls back,
the second transaction doesn't use *dirty (uncommitted)* data.

🧩 Example 1 – Recoverable Schedule

Step	Transaction	Operation
1	T1	Write(X)
2	T2	Read(X) ← value written by T1
3	T1	Commit
4	T2	Commit

✅ Here T2 reads data written by T1,
and **T2 commits only after T1 commits**,
so this is a **Recoverable Schedule**.

❌ Example 2 – Non-Recoverable Schedule

Step	Transaction	Operation
1	T1	Write(X)
2	T2	Read(X) ← value written by T1
3	T2	Commit
4	T1	Rollback

❌ Here T2 commits **before** T1,
but later T1 **rolls back** —
this means T2 has used **invalid (dirty)** data.
Hence, this is a **Non-Recoverable Schedule**.

🔷 Why Recoverability is Desirable

Recoverability ensures that:

1. **Database consistency** is maintained even after failures.
2. **Cascading rollbacks** and **dirty reads** are avoided.
3. Transactions that have committed are based on **valid data**.
4. **Atomicity** and **Durability (A, D of ACID)** are preserved.

If a schedule is **not recoverable**,
then some committed transactions may depend on **aborted** ones,
which leads to **inconsistency** in the database.

🔹 When could a Non-Recoverable Schedule be allowed?

In general, **non-recoverable schedules are avoided**,
but they **might be allowed** in **very rare cases** such as:

1. **Read-only transactions:**
   If transactions **only read data** and do not update anything,
   then recoverability is not an issue.
2. **Temporary / test systems:**
   In **real-time or experimental systems**,
   where performance is more important than consistency,
   we may allow non-recoverable schedules for **speed**.
3. **Unimportant data:**
   When transactions work on **non-critical data** (e.g., cached or temporary values),
   consistency may not be strictly enforced.

🔹 Summary Table

Schedule Type	Description	Safe?
Recoverable	Ti commits after all transactions whose data it read have committed	✅ Yes
Non-Recoverable	Ti commits before transactions whose data it read have committed	❌ No

✅ **In short:**

> A **recoverable schedule** ensures no committed transaction depends on an **uncommitted** one.
> It is **desirable** because it maintains **database consistency**.
> Non-recoverable schedules are rarely used — only when **speed is prioritized over safety**.

20) Check whether following schedule is view serializable or not. Justify your answer. (Note : T1 & T2 are transactions). Also explain the concept of view equivalent schedules and conflict equivalent schedule considering the example schedule given below :

T_1	T_2
read (A)	
A := A – 50	
	read (A)
	temp :=A * 0.1
	A := A - temp
	write (A)
	read (B)
write (A)	
read (B)	
B := B + 50	
write (B)	
	B := B + temp
	write (B)

06/06/2023 CEGP013091 10.248.216.238 06/06/2023 10:08:17 static-238

1. Transcribe the schedule (sequence)

Let initial values be `A0`, `B0`. The operations (in order) are:

1. `T1: r1(A)` (T1 reads A → reads `A0`)
2. `T1: A := A - 50` (computes new value, not yet written)
3. `T2: r2(A)` (T2 reads A → reads `A0`)
4. `T2: temp := A * 0.1`
5. `T2: A := A - temp`
6. `T2: w2(A)` (T2 writes A)
7. `T2: r2(B)` (T2 reads B → reads `B0`)
8. `T1: w1(A)` (T1 writes A, overwrites T2's A)
9. `T1: r1(B)` (T1 reads B → reads `B0`)
10. `T1: B := B + 50`
11. `T1: w1(B)` (T1 writes B = `B0 + 50`)
12. `T2: B := B + temp`
13. `T2: w2(B)` (T2 writes final B = `B0 + 50 + temp`)

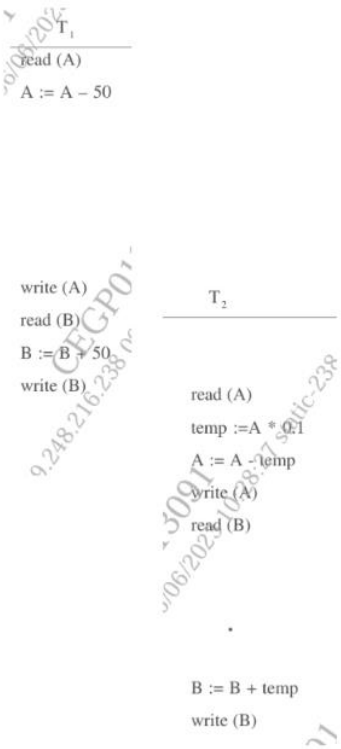
Final values: `A` is the value written by `T1` (step 8), `B` is final value written by `T2` (step 13).

2. View-equivalence — definition (short)

Two schedules `S` and `S'` are **view equivalent** if for every data item `X`:

1. **Initial-read rule:** A transaction that reads the initial value of `X` in `S` must also read the initial value in `S'`.
2. **Read-from rule:** If in `S`, `Ti` reads `X` from `Tj` (i.e., `Tj` wrote the value that `Ti` read), then in `S'` `Ti` must also read `X` from `Tj`.
3. **Final-write rule:** The transaction that does the **final write** of `X` in `S` must also do the final write of `X` in `S'`.

If a schedule is view-equivalent to a serial schedule, it is **view-serializable**.



3. Test view-serializability (compare with two possible serial orders)

Only two serial orders for two transactions: **T1 → T2** or **T2 → T1**. We check both.

Check serial order T1 → T2

- In serial `T1→T2`: `T1` runs first and writes A (so `T2` would read A **after** `T1`'s write). So `T2` would see `A` = value produced by `T1`.
- In our schedule: `T2` read `A0` at step 3 (before `T1` wrote A at step 8). So `T2` did **not** read the value written by `T1`.
  → **Read-from relation differs** → Not view-equivalent to `T1→T2`.

Check serial order T2 → T1

- In serial `T2→T1`: `T2` runs first and writes A, then `T1` would read A **after** `T2`'s write (so `T1` would read the value written by `T2`).
- In our schedule: `T1` read `A0` at step 1 (before `T2` wrote A). So `T1` did **not** read the value written by `T2`.
  → **Read-from relation differs** → Not view-equivalent to `T2→T1`.

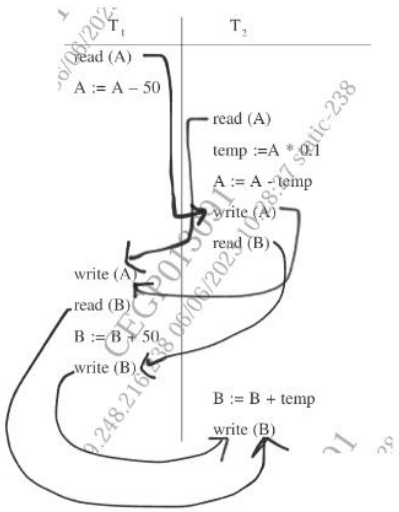
Since the schedule is not view-equivalent to either serial order, the schedule is **NOT view-serializable**.

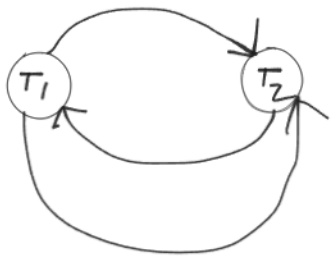
(Also final-write rule: final writer of `A` in schedule is `T1`. In `T2→T1` final writer for `A` would be `T1` — that matches; but other read-from conditions already fail. For `B`, final writer is `T2`, which conflicts with `T2→T1` final-write expectation; so final-write rule also fails for some serial orders.)

4. Conflict-equivalence (conflict-serializability) — definition (short)

Two schedules are **conflict equivalent** if one can be transformed into the other by swapping adjacent **non-conflicting** operations.
A convenient test: build the **precedence graph** (nodes = transactions). For every conflicting pair (different transactions, same data item, at least one is write), add a directed edge `Ti → Tj` if an operation of `Ti` appears before a conflicting operation of `Tj`.
If the graph has a **cycle**, the schedule is **not conflict serializable**.

Conflicts





NOT Conflict
Serial....

5. Find conflicts in the given schedule (step-by-step)

Conflicts on data item A

- `r1(A)` (step1) before `w2(A)` (step6) → **R–W conflict** ⇒ edge `T1 → T2`.
- `r2(A)` (step3) before `w1(A)` (step8) → **R–W conflict** ⇒ edge `T2 → T1`.
- `w2(A)` (step6) before `w1(A)` (step8) → **W–W conflict** ⇒ edge `T2 → T1`.

So for A we have both `T1 → T2` and `T2 → T1` (i.e., mutual edges → cycle).

Conflicts on data item B

- `r2(B)` (step7) before `w1(B)` (step11) → **R–W conflict** ⇒ edge `T2 → T1`.
- `w1(B)` (step11) before `w2(B)` (step13) → **W–W conflict** ⇒ edge `T1 → T2`.
- `r1(B)` (step9) before `w2(B)` (step13) → **R–W conflict** ⇒ edge `T1 → T2`.

B produces edges both `T2 → T1` and `T1 → T2` as well.

Precedence graph summary:

- Edges `T1 → T2` (from A: r1→w2; from B: w1→w2, r1→w2)
- Edges `T2 → T1` (from A: r2→w1 and w2→w1; from B: r2→w1)

Graph contains a **cycle** (`T1 → T2 → T1`).

Therefore the schedule is **NOT conflict serializable**.

6. Conclusion (clear)

- The schedule is **not view-serializable** (fails read-from / initial-read / final-write tests).
- The schedule is **not conflict-serializable** (precedence graph has a cycle).
  So the given schedule **is neither view-serializable nor conflict-serializable**.

---

7. Intuition / short reason (why it fails)

Both transactions read the initial value of `A` and then write `A` in overlapping order (each read occurs before the other's write), creating cross dependencies. These cross read–write and write–write interactions produce mutual ordering constraints (edges both ways) and therefore a cycle — hence no serial order can match both read/write relationships.

21) What is conflict serializability? Check following schedule is conflict serializable or not? Also, explain the concept of conflict equivalent schedule.

T1	T2	T3	T4
R(X)			
R(Z)			
	W(X)		
		R(Y)	
		W(Y)	
			W(X)
			W(Y)
			W(Z)

CEC 49.248.216.238 16/05/

1. What is Conflict Serializability (short)

A schedule is **conflict serializable** if it is **conflict equivalent** to some serial schedule.

Practical test: build the **precedence (serialization) graph** — one node per transaction; add an edge `Ti → Tj` when an operation of `Ti` that **conflicts** with an operation of `Tj` appears **earlier** in the schedule.

- If the graph has **no cycle** → schedule is **conflict serializable**.
- If the graph has a **cycle** → **not** conflict serializable.

> **Conflict** = two operations from different transactions on the **same data item** and **at least one** is a **write** (R–W, W–R, or W–W).

---

2. Transcribe the given schedule (top → bottom, row by row)

From the table the operations occur in this order:

1. `T1 : r1(X)`
2. `T1 : r1(Z)`
3. `T2 : w2(X)`
4. `T3 : r3(Y)`
5. `T3 : w3(Y)`
6. `T4 : w4(X)`
7. `T4 : w4(Y)`
8. `T4 : w4(Z)`

(If you prefer, think of these as the time-ordered steps 1..8.)

3. Find all conflicts (same item, different transactions, at least one write)

On X

- `r1(X)` before `w2(X)` → **R–W** conflict ⇒ edge **T1 → T2**.
- `w2(X)` before `w4(X)` → **W–W** conflict ⇒ edge **T2 → T4**.
- `r1(X)` before `w4(X)` → **R–W** conflict ⇒ edge **T1 → T4**.

On Y

- `w3(Y)` before `w4(Y)` → **W–W** conflict ⇒ edge **T3 → T4**.
  (`r3(Y)` and `w3(Y)` are in same transaction T3 so no conflict there.)

On Z

- `r1(Z)` before `w4(Z)` → **R–W** conflict ⇒ edge **T1 → T4** (already noted).

4. Precedence (serialization) graph

Nodes: `T1, T2, T3, T4`

Edges collected:

- `T1 → T2`
- `T2 → T4`
- `T1 → T4`
- `T3 → T4`

Graph has no edge that points back to `T1`, `T2`, or `T3` (i.e. **no cycle**).

Visual (simple):

```lua
T1 → T2 → T4
 \          ^
  \->------|
T3 ---->---|
```

(Edges into `T4` from `T1`, `T2`, `T3`.)

5. Conclusion — Is schedule conflict serializable?

Yes.

The precedence graph is **acyclic**, so the schedule is **conflict serializable**.

One valid equivalent serial order (consistent with the edges) is:

T1 → T2 → T3 → T4 (or T3 can appear before or after T2 as long as both are before T4 and T1 is before T2 & T4).

The only required orderings are those given by edges: `T1` must precede `T2` and `T4`; `T2` must precede `T4`; `T3` must precede `T4`.

---

6. Short note — What is a conflict-equivalent schedule?

Two schedules are **conflict-equivalent** if you can transform one into the other by repeatedly swapping **adjacent** operations that **do not conflict** (i.e., operations on different items or both reads).

If one schedule is conflict-equivalent to a serial schedule, it is conflict-serializable. The precedence graph test is the standard way to check this.

UNIT 5

22) Explain how NOSQL databases are different than relational databases? Describe in detail the key value store NOSQL data model with example.

🔹 1. Introduction

Databases are mainly divided into two types:

- **Relational Databases (RDBMS)** — like MySQL, Oracle, PostgreSQL
- **NoSQL Databases** — like MongoDB, Redis, Cassandra

The term **NoSQL** means "*Not Only SQL*".
It provides a way to store and retrieve data that is not modeled in traditional table formats.

🔹 2. Difference between Relational and NoSQL Databases

Feature	Relational Database (RDBMS)	NoSQL Database
Data Model	Tables (rows & columns)	Key-Value, Document, Column, Graph
Schema	Fixed schema (predefined)	Dynamic / schema-less
Scalability	Vertical scaling (increase server capacity)	Horizontal scaling (add more servers)
Query Language	SQL (Structured Query Language)	No fixed query language (JSON, APIs, etc.)
Transactions	Supports ACID (Atomicity, Consistency, Isolation, Durability)	Often BASE (Basically Available, Soft state, Eventual consistency)
Use Case	Suitable for structured data and complex queries	Suitable for unstructured or rapidly changing data
Examples	MySQL, PostgreSQL, Oracle	MongoDB, Redis, Cassandra, DynamoDB

🔹 3. Key-Value Store NoSQL Data Model

Definition:

In a **Key-Value Store**, data is stored as a pair — **(key, value)**.

- The **key** is a unique identifier (like a primary key).
- The **value** can be any type of data — string, JSON object, image, etc.

It is the **simplest type of NoSQL database**.

🔹 4. Structure

```lua
Key        →        Value
-------------------------
user101    →  {"name": "Karan", "age": 22, "city": "Pune"}
user102    →  {"name": "Sahil", "age": 23, "city": "Mumbai"}
```

Here:

- Each **key** uniquely identifies a record.
- Each **value** stores the actual data.

🔹 5. Example (Redis / DynamoDB)

Example in Redis (a Key-Value NoSQL database):

```bash
SET user101 "{'name':'Karan', 'age':22, 'city':'Pune'}"
GET user101
```

Output:

```bash
{'name':'Karan', 'age':22, 'city':'Pune'}
```

👉 Here,

- `SET` command stores the value with key `user101`.
- `GET` command retrieves the value using that key.

🔹 6. Advantages

✅ Very fast for read/write operations
✅ Simple to design and use
✅ Easily scalable across many servers
✅ Ideal for caching, session management, user profiles, etc.

🔹 7. Limitations

❌ No support for complex queries or relationships
❌ No data validation (since schema-less)
❌ Limited transactional support

🔹 8. Applications

- Caching (e.g., Redis for website caching)
- Shopping carts
- Gaming leaderboards
- User session management

🟩 Summary

Aspect	Key-Value NoSQL Model
Storage	Key-Value pairs
Schema	Schema-less
Performance	Extremely fast
Best For	Simple lookups and high-speed data access
Examples	Redis, DynamoDB, Riak

Final Answer Summary (for exam):
NoSQL databases differ from relational databases as they are schema-less, horizontally scalable, and handle unstructured data.
In the **Key-Value model**, each record is stored as a unique key with its corresponding value, allowing fast data access.
Examples include Redis and Amazon DynamoDB.

23) Explain BASE properties with its significance. How soft state of system is depending on Eventual consistency property?

🔹 1. Introduction

Traditional databases (RDBMS) follow **ACID** properties for consistency and reliability.
However, **NoSQL databases** (like Cassandra, DynamoDB, MongoDB, etc.) are designed for scalability and high availability — they follow **BASE** properties instead of ACID.

🔹 2. Meaning of BASE

BASE stands for:

- **B** → *Basically Available*
- **A** → *Soft State*
- **SE** → *Eventual Consistency*

It provides a flexible model for distributed systems where strict consistency is relaxed for higher availability and fault tolerance.

🔹 3. BASE Properties in Detail

Property	Meaning	Significance / Explanation
Basically Available	The system guarantees **availability** of data even in case of partial failure.	Even if some nodes fail, the database can still respond to requests (maybe with slightly outdated data).
Soft State	The system's state may change over time even without input.	Because data replicas are being updated asynchronously, the system does not need to be in a consistent state all the time.
Eventual Consistency	The system will become consistent **after some time** when all updates have been propagated.	All replicas will eventually hold the same data once all updates are completed.

🔹 4. Relationship between Soft State and Eventual Consistency

- **Soft State** means the system's data can temporarily be **inconsistent** — because updates are happening in different nodes at different times.
- This temporary inconsistency is **acceptable** because the **Eventual Consistency** property guarantees that, **after some time**, all nodes will synchronize and have the same data.

🟩 **Example:**

Suppose you update your profile picture on a social media app.

- At first, some users (on one server) still see your old picture (soft state — temporary inconsistency).
- After a few seconds, all servers update (eventual consistency).
- Finally, everyone sees the same updated picture — the system becomes consistent again.

So, **Soft State depends on Eventual Consistency** because it assumes that temporary inconsistencies are okay as long as the system eventually becomes consistent.

🔹 5. Comparison between ACID and BASE

Aspect	ACID (RDBMS)	BASE (NoSQL)
Focus	Consistency and reliability	Availability and scalability
Consistency	Immediate consistency	Eventual consistency
State	Rigid / stable	Soft / changing
System type	Centralized	Distributed
Use cases	Banking, transactions	Social media, IoT, Big Data apps

🔹 6. Significance of BASE

✅ Increases system **availability**
✅ Supports **high scalability** and **fault tolerance**
✅ Suitable for **large distributed systems**
✅ Provides **better performance** and **low latency**

🟩 7. Summary

- **BASE** is used in NoSQL systems to ensure high availability.
- **Soft State** allows temporary inconsistency.
- **Eventual Consistency** ensures that the system will finally reach a consistent state.
- Hence, *Soft State depends on Eventual Consistency*, because the system is temporarily inconsistent until it becomes consistent again.

Final Short Answer (for 2–3 marks summary section):

> BASE stands for *Basically Available*, *Soft State*, *Eventual Consistency*.
> It ensures high availability and scalability in NoSQL systems.
> The *Soft State* allows temporary inconsistency, which depends on the *Eventual Consistency* property to finally make all replicas consistent over time.

24) List the different NOSQL data models. Explain document store NOSQL data model with example.

🔹 1. Introduction

- **NoSQL (Not Only SQL)** databases are designed to handle **large amounts of unstructured and semi-structured data**.
- They provide **high performance**, **scalability**, and **flexibility**, which are difficult to achieve in traditional relational databases.

🔹 2. Different Types of NoSQL Data Models

Sr. No.	Type of NoSQL Model	Description	Examples
1	**Key-Value Store**	Data stored as key-value pairs (like a dictionary or map).	Redis, Amazon DynamoDB
2	**Document Store**	Data stored as documents (JSON, BSON, or XML format).	MongoDB, CouchDB
3	**Column-Family Store**	Data stored in columns instead of rows (optimized for analytics).	Cassandra, HBase
4	**Graph-Based Store**	Data stored as nodes and relationships (edges).	Neo4j, Amazon Neptune

🔹 3. Document Store NoSQL Data Model

📘 Definition:

In a **Document Store NoSQL database**, data is stored in the form of **documents**, which are typically in **JSON** or **BSON** format.

Each document contains fields and values, similar to how objects are represented in programming languages.

- Each **document** is a **self-contained record** that includes both data and its structure.
- Documents are grouped into **collections** (like tables in RDBMS).

🔹 4. Structure Example

Let's consider a **student database**.

Each student's data can be stored as one document:

```json
{
  "student_id": 101,
  "name": "Karan Salunkhe",
  "course": "Computer Engineering",
  "subjects": ["DBMS", "AI", "CN"],
  "marks": {
    "DBMS": 85,
    "AI": 90,
    "CN": 88
  }
}
```

Here:

- `student_id`, `name`, and `course` are fields.
- `subjects` is an **array**.
- `marks` is a **nested object** (key-value pairs).

🔹 5. Features of Document Store Model

1. **Schema-less** → No fixed structure; new fields can be added anytime.
2. **Self-describing documents** → Each document contains all related data.
3. **Nested data supported** → Arrays and objects can be embedded.
4. **Fast data retrieval** → Data stored in documents can be fetched quickly.
5. **Easier to scale horizontally** → Suitable for distributed systems.

🔹 6. Advantages

✅ Flexible and easy to modify structure
✅ High performance for read/write operations
✅ Supports complex data types
✅ Natural fit for object-oriented applications

🔹 7. Example Using MongoDB (Document Store Database)

➤ Insert a document:

```javascript
db.students.insert({
  student_id: 101,
  name: "Karan",
  course: "Computer Engineering",
  marks: { DBMS: 85, AI: 90 }
})
```

➤ Query a document:

```javascript
db.students.find({ name: "Karan" })
```

MongoDB stores this data as a **document** within a **collection** called `students`.

🔹 8. Comparison with RDBMS

Aspect	RDBMS	Document Store NoSQL
Structure	Tables and rows	Collections and documents
Schema	Fixed	Flexible (schema-less)
Query Language	SQL	JSON-like queries
Data Format	Tabular	JSON / BSON
Relationships	Using joins	Embedded documents

🔹 9. Use Cases

- Content management systems
- E-commerce catalogs
- User profile storage
- Blogs and social media apps

25) State and explain the concept of CAP theorem and BASE properties with example.

🔷 1. Introduction

When working with **distributed databases** (like NoSQL systems), it's difficult to guarantee all desirable properties like consistency, availability, and partition tolerance at the same time.

To explain this limitation, **Eric Brewer** proposed the **CAP Theorem**, and **BASE properties** were introduced to complement it for NoSQL databases.

🧠 2. CAP Theorem

Definition:

> The **CAP theorem** states that a **distributed database system** can provide only **two** out of the following three properties at the same time —
> **Consistency, Availability, and Partition tolerance.**

3. Explanation of Each Property

Property	Meaning	Example / Explanation
Consistency (C)	Every read receives the **most recent write** or an **error**.	If user updates a profile picture, all users immediately see the new picture.
Availability (A)	Every request receives a **response**, even if it's not the latest data.	System always gives a response, even if some data is slightly outdated.
Partition Tolerance (P)	The system continues to function even if **network failures** (partitions) occur between nodes.	If some servers can't communicate, the system still works for the rest.

4. CAP Theorem Triangle Diagram

mathematica Copy code

```
              Consistency
                 /\
                /  \
               /    \
     Partition  /_____\  Availability
              Tolerance
```

- A system can **choose any two**:
  - **CP**: Consistency + Partition Tolerance (e.g., MongoDB in some configs)
  - **AP**: Availability + Partition Tolerance (e.g., Cassandra, DynamoDB)
  - **CA**: Consistency + Availability (only possible in single-node systems)

5. Example:

- Suppose you use a banking app.
  - If it chooses **Consistency + Partition Tolerance (CP)** — when servers are disconnected, some transactions may wait (no immediate access) to maintain correct balances.
  - If it chooses **Availability + Partition Tolerance (AP)** — it allows transactions even during a network issue, but data might not be updated instantly everywhere.

⚙️ 6. BASE Properties

The **BASE** properties were introduced for **NoSQL databases** to relax strict ACID rules (used in traditional RDBMS) and achieve better scalability and performance in distributed systems.

Example of BASE:

Consider a **social media app**:

- You update your profile picture.
- For a few seconds, some friends still see your old picture (**soft state**).
- After a short delay, everyone sees the new one (**eventual consistency**).
- The system was **basically available** during the update.

---

7. Relationship Between CAP and BASE

CAP Aspect	BASE Relation
Consistency	Eventual Consistency (relaxed)
Availability	Basically Available
Partition Tolerance	Supported through distributed design

➡️ **BASE** is a **practical approach** used by NoSQL databases to handle **CAP trade-offs**, focusing more on availability and partition tolerance than strict consistency.

8. Comparison: ACID vs BASE

Feature	ACID (SQL)	BASE (NoSQL)
Focus	Reliability & Consistency	Scalability & Availability
Consistency Type	Immediate	Eventual
Transaction	Strict & atomic	Flexible
Use Cases	Banking, Finance	E-commerce, Social Media

🟩 9. Summary

- **CAP Theorem:** A distributed system can guarantee only **two** of **Consistency**, **Availability**, **Partition Tolerance** at a time.
- **BASE Properties:** Provide a **flexible**, **highly available**, and **scalable** approach for NoSQL databases.
- Together, they explain how modern distributed databases trade strict consistency for better **performance** and **reliability**.

26) Enlist the different types of NOSQL databases and explain with suitable examples.

(1) Key–Value Store

Definition:

- Simplest type of NoSQL database.
- Each record (called an item) consists of a **unique key** and a **value**.

Example:

Imagine a shopping cart application:

Key	Value
user101_cart	{"item": "Shoes", "qty": 2, "price": 2000}
user102_cart	{"item": "Watch", "qty": 1, "price": 1500}

Databases:

- Redis, Riak, Amazon DynamoDB

Use Case:

- Caching, session management, and fast lookups.

(2) Document Store

Definition:

- Stores data as **documents** in JSON, BSON, or XML format.
- Each document contains key-value pairs, arrays, or nested documents.
- Flexible (schema-less) and supports complex data.

Example:

```json
{
  "student_id": 101,
  "name": "Karan",
  "course": "Computer Engineering",
  "marks": { "DBMS": 85, "AI": 90 }
}
```

Databases:

- MongoDB, CouchDB

Use Case:

- E-commerce catalogs, user profiles, blogs, and CMS systems.

(3) Column-Family Store

Definition:

- Data is organized into rows and columns, but columns are grouped into families.
- Each row can have different columns (flexible schema).
- Optimized for large-scale analytical queries.

Example:

Row Key	Personal	Academic
101	name: Karan	marks: 85
102	name: Sahil	marks: 90

Databases:

- Apache Cassandra, HBase

Use Case:

- Big data analytics, time-series data, recommendation systems.

(4) Graph-Based Store

Definition:

- Designed to store relationships between entities as graphs.
- Data is stored as:
  - Nodes → entities (e.g., users, places)
  - Edges → relationships (e.g., "follows", "likes")
  - Properties → attributes of nodes or edges.

Example:

```scss
(User: Karan) –[follows]→ (User: Sahil)
```

Databases:

- Neo4j, Amazon Neptune

Use Case:

- Social networks, recommendation engines, fraud detection.

🔹 4. Comparison Table

Type	Data Format	Schema	Best For
Key–Value	Key: String, Value: Any	Schema-less	Caching, sessions
Document	JSON / BSON	Flexible	User data, CMS
Column-Family	Column sets	Semi-structured	Analytics, Big data
Graph	Nodes and Edges	Flexible	Relationships, networks

27) What is structured and unstructured data. Explain with example.

🔹 2. Comparison Between Structured and Unstructured Data

Feature	Structured Data	Unstructured Data
Definition	Data that is **organized** and stored in a fixed format (like rows and columns).	Data that has **no predefined structure** or format.
Storage Format	Stored in **tables** (rows and columns) in relational databases.	Stored as **files, documents, images, videos,** etc.
Schema	Has a **predefined schema** (fixed structure).	**No fixed schema**; format can vary.
Ease of Storage	Easy to store, search, and analyze using SQL.	Difficult to store and analyze; requires special tools.
Data Model	Relational model (RDBMS).	Non-relational (NoSQL, file systems).
Examples	Employee records, student marks, bank transactions, product details.	Emails, images, social media posts, audio, video files.
Tools/Databases Used	MySQL, Oracle, PostgreSQL, MS SQL Server.	MongoDB, Cassandra, Hadoop, ElasticSearch.
Query Language	Can be queried using **SQL**.	Uses **NoSQL** or data processing tools (like Spark, MapReduce).
Scalability	Vertically scalable (add resources to one server).	Horizontally scalable (add more servers).
Data Type	Quantitative (numbers, text).	Qualitative (media files, free text).

🔹 3. Examples

📄 Structured Data Example (RDBMS Table):

Emp_ID	Name	Dept	Salary
101	Karan	IT	60,000
102	Sahil	HR	55,000

- Data is organized in rows and columns.
- Easy to query using SQL:

```sql
SELECT * FROM Employee WHERE Dept = 'IT';
```

🖼️ Unstructured Data Example:

File Name	Type	Description
photo1.jpg	Image	Employee photo
feedback.mp3	Audio	Customer feedback
post.txt	Text	User social media post

- Data is not organized in tabular form.
- Cannot be queried easily using SQL.
- Requires tools like **Hadoop** or **NoSQL** databases to analyze.

🔹 4. Semi-Structured Data (Bonus Point)

> There is also **semi-structured data**, which lies between structured and unstructured data.
> Example: **JSON**, **XML**, **CSV** files.

```json
{
  "student_id": 101,
  "name": "Karan",
  "subjects": ["DBMS", "AI", "CN"]
}
```

28) Analyze the use of NOSQL databases in current social networking environment also explain need of NOSQL databases in social networking environment over RDBMS.

🔹 1. Introduction

Social networking platforms like **Facebook, Instagram, Twitter (X), LinkedIn**, and **Snapchat** handle **massive amounts of user-generated data** — posts, comments, likes, images, videos, and messages.

Traditional **Relational Databases (RDBMS)** are **not suitable** for handling such **large-scale, unstructured, and rapidly changing** data.

Hence, **NoSQL databases** are used because they are **highly scalable, flexible, and efficient** for managing social media data.

🔹 2. Need of NoSQL Databases in Social Networking Environment

Challenges in Social Media	How NoSQL Solves It
1. Large Volume of Data (millions of posts and images daily)	NoSQL databases like **Cassandra** or **MongoDB** can scale horizontally across multiple servers.
2. Unstructured / Semi-structured Data (posts, images, comments, hashtags)	NoSQL can store **JSON, documents, key-value pairs**, and other non-tabular formats easily.
3. High Read/Write Speed (real-time likes, shares, notifications)	NoSQL provides **high throughput** using distributed data storage.
4. Frequent Schema Changes (adding new features like reels, stories, reactions)	NoSQL databases are **schema-less**, so new data fields can be added easily.
5. Global User Base (millions of concurrent users)	NoSQL supports **replication** and **sharding**, ensuring fast access worldwide.
6. Real-time Analytics (trending topics, user suggestions)	NoSQL databases allow **fast aggregation and real-time queries**.

🔹 3. Use of NoSQL Databases in Social Networking

Platform	NoSQL Database Used	Purpose / Functionality
Facebook	Cassandra, HBase	Storing messages, posts, and user feed.
Instagram	Cassandra, Redis	Managing likes, followers, and timeline feeds.
Twitter (X)	Redis, MySQL (hybrid), Cassandra	Caching tweets, timelines, and user sessions.
LinkedIn	Voldemort, Espresso	Handling professional feeds and messages.
Pinterest	Redis, HBase	Managing pins, boards, and images efficiently.

🔹 4. Advantages of NoSQL over RDBMS in Social Media

Feature	RDBMS (Traditional)	NoSQL (Modern)
Data Model	Fixed rows & columns	Flexible document, graph, or key-value models
Scalability	Vertical (limited)	Horizontal (add more servers)
Schema	Rigid schema	Schema-less
Data Type	Structured only	Handles structured, semi-structured & unstructured
Performance	Slower for big data	High performance under heavy load
Replication & Sharding	Limited support	Built-in support
Use Case Fit	Small/medium data, transactions	Big Data, analytics, social feeds

🔹 6. Significance / Benefits Summary

- Handles **large-scale, real-time user data**
- Provides **high availability** and **fault tolerance**
- Supports **distributed architecture**
- Offers **flexible data models** for dynamic applications
- Enables **real-time analytics** for trends and engagement

🟩 7. Conclusion

> NoSQL databases are essential for social networking environments due to their ability to handle **massive, unstructured, and rapidly changing data**.
> They provide **better scalability, flexibility, and speed** compared to traditional RDBMS, making them ideal for platforms like Facebook, Twitter, and Instagram.

29) Explain the CRUD operations used in MongoDB with example.

🔹 1. Introduction

- MongoDB is a NoSQL document-oriented database that stores data in JSON-like documents (BSON format).
- The basic operations that can be performed on data are called CRUD operations.

> CRUD stands for **Create, Read, Update, and Delete** — the four basic functions of persistent storage.

🔹 2. CRUD Operations in MongoDB

Operation	Meaning	MongoDB Command
C – Create	Insert new documents (records) into a collection.	`insertOne()` / `insertMany()`
R – Read	Retrieve documents from a collection.	`find()`
U – Update	Modify existing documents.	`updateOne()` / `updateMany()`
D – Delete	Remove documents from a collection.	`deleteOne()` / `deleteMany()`

🟢 (A) CREATE Operation

Used to insert new records.

Example:

```js
db.students.insertOne({
   name: "Karan",
   age: 21,
   course: "B.Tech",
   city: "Pune"
})
```

Multiple insert:

```js
db.students.insertMany([
   { name: "Sahil", age: 22, course: "BCA" },
   { name: "Meena", age: 20, course: "B.Sc" }
])
```

🔵 (B) READ Operation

Used to fetch or view data from a collection.

Example:

```js
db.students.find()
```

➡️ Displays all student records.

Example with condition:

```js
db.students.find({ city: "Pune" })
```

➡️ Displays all students from Pune.

Example with projection:

```js
db.students.find({}, { name: 1, course: 1 })
```

➡️ Shows only name and course fields.

🟡 (C) UPDATE Operation

Used to modify existing data.

Example:

```js
db.students.updateOne(
   { name: "Karan" },
   { $set: { city: "Mumbai" } }
)
```

➡️ Updates the city of the student named Karan.

Update multiple documents:

```js
db.students.updateMany(
   { course: "BCA" },
   { $set: { course: "BCA - Computer Science" } }
)
```

🔴 (D) DELETE Operation

Used to remove documents from a collection.

Example:

```js
db.students.deleteOne({ name: "Sahil" })
```

➡️ Deletes the document where the name is Sahil.

Delete multiple documents:

```js
db.students.deleteMany({ city: "Mumbai" })
```

➡️ Deletes all documents of students from Mumbai.

30) Explain Map Reduce with example.

🔹 1. Introduction

- **MapReduce** is a **data processing model** introduced by **Google** for handling **large-scale data** on distributed systems (like Hadoop).
- It is used to **process big data in parallel** across many computers (nodes) in a cluster.
- The main goal is to divide a big task into smaller sub-tasks, process them in parallel, and combine the results.

🔹 2. Phases of MapReduce

MapReduce works in **two main phases**:

Phase	Description
Map Phase	The input data is divided into chunks. Each chunk is processed by a "Mapper" function, which converts data into **(key, value)** pairs.
Reduce Phase	The "Reducer" function combines all values belonging to the same key and performs an aggregation or computation (like sum, count, average, etc.).

🔹 3. Working Steps

1. **Input Splitting** – Data is split into multiple parts.
2. **Mapping** – Each mapper processes its data and generates intermediate key-value pairs.
3. **Shuffling & Sorting** – All key-value pairs are grouped by key.
4. **Reducing** – Reducer combines all values with the same key to produce the final result.
5. **Output** – The final processed data is stored in HDFS (Hadoop Distributed File System).

🔹 4. Example: Word Count Problem

Problem: Count how many times each word appears in a document.

Input Data

```bash
File 1: "cat dog cat"
File 2: "dog tiger cat"
```

Step 1 – Map Phase

Each word is mapped to a key-value pair (word, 1):

Mapper Output
(cat, 1)
(dog, 1)
(cat, 1)
(dog, 1)
(tiger, 1)
(cat, 1)

Step 2 – Shuffle & Sort

Group values by the same key:

Word (Key)	Values
cat	[1, 1, 1]
dog	[1, 1]
tiger	[1]

Step 3 – Reduce Phase

Combine values for each key (sum them):

Reducer Output
(cat, 3)
(dog, 2)
(tiger, 1)

Step 4 – Final Output

```bash
cat → 3
dog → 2
tiger → 1
```

🔷 5. Advantages of MapReduce

- ✅ Handles **large datasets efficiently**
- ✅ Provides **parallel and distributed processing**
- ✅ Offers **fault tolerance** (failed tasks automatically re-executed)
- ✅ Scalable to hundreds or thousands of machines

🔷 6. Conclusion

> MapReduce simplifies **large-scale data processing** by breaking it into smaller sub-tasks (Map) and combining their results (Reduce).
> It forms the **core concept behind big data tools like Hadoop**.

31) Explain Structured, Semi-structured and Unstructured data types with examples.

Type of Data	Description	Storage Format / Structure	Examples	Used In
1. Structured Data	Data that is **organized in fixed fields** like rows and columns. It follows a **specific schema** and is easy to store, search, and analyze.	Tables (rows & columns) in databases like **RDBMS** (SQL).	• Student database table • Employee records • Banking transactions	Relational Databases (e.g., MySQL, Oracle, PostgreSQL)
2. Semi-Structured Data	Data that **does not follow a strict schema** but still has some organizational elements (like tags or keys). It allows **flexibility** in structure.	Stored in **key-value** or **document** formats such as **JSON, XML, YAML**.	• JSON documents • XML files • Email (header + body)	NoSQL databases (e.g., MongoDB, CouchDB)
3. Unstructured Data	Data with **no predefined format or structure**. It's difficult to organize and analyze directly.	Stored as files (text, images, videos, audio). No schema or pattern.	• Social media posts • Images, Videos • PDFs, Audio files	Big Data systems, Data Lakes, Cloud Storage

🔹 Summary

Category	Schema	Ease of Querying	Flexibility
Structured	Fixed schema	Easy (SQL)	Low
Semi-Structured	Partial / Flexible schema	Moderate	Medium
Unstructured	No schema	Difficult	High

Example Visualization

Data Type	Example
Structured	`{"Student_ID": 1, "Name": "Karan", "Marks": 85}` *(Stored in a table)*
Semi-Structured	`<student><name>Karan</name><marks>85</marks></student>` *(XML / JSON)*
Unstructured	An image file `Karan.jpg` or a video `Lecture.mp4` *(no defined fields)*

Conclusion:

> Structured data is easy to manage but rigid,
> Semi-structured data provides flexibility,
> and Unstructured data represents the **majority of today's big data** used in social media, IoT, and AI systems.

32) Describe the following operations with MongoDB syntax: i) Map-Reduce ii) Aggregation pipeline

i) Map-Reduce in MongoDB

🔹 Definition

- **Map-Reduce** is a **data processing technique** in MongoDB used to perform **complex data aggregation and analysis** on large collections.
- It uses **two JavaScript functions** — `map` and `reduce`.

🔹 Functions Used

Function	Purpose
Map function	Processes each document and emits **key-value pairs**.
Reduce function	Combines values that have the same key and returns a single result.

🔹 Syntax

js Copy code

```js
db.collection.mapReduce(
   function() { emit(key, value); },        // Map function
   function(key, values) { return reducedValue; }, // Reduce function
   {
      out: "outputCollection"
   }
)
```

🔹 Example: Count number of students per course

```js
db.students.mapReduce(
   function() { emit(this.course, 1); },       // Map: key = course, value = 1
   function(key, values) { return Array.sum(values); }, // Reduce: sum values
   { out: "course_count" }
)
```

Output (in `course_count` collection):

```json
{ "_id": "BCA", "value": 5 }
{ "_id": "B.Tech", "value": 3 }
{ "_id": "B.Sc", "value": 2 }
```

🔹 Advantages

✅ Handles large data efficiently
✅ Supports parallel execution
✅ Useful for aggregation, filtering, and statistical analysis

---

ii) Aggregation Pipeline in MongoDB

🔹 Definition

- **Aggregation Pipeline** is a **framework for data aggregation** that processes documents in multiple **stages (like a pipeline)**.
- Each stage transforms the data and passes it to the next stage.

🔹 Common Stages

Stage	Description
`$match`	Filters documents (like WHERE in SQL).
`$group`	Groups documents by a specified field and performs aggregation.
`$sort`	Sorts the documents.
`$project`	Selects specific fields or creates new computed fields.
`$limit` / `$skip`	Controls number of documents returned.

🔹 Syntax

```js
db.collection.aggregate([
   { stage1 },
   { stage2 },
   ...
])
```

🔹 Example: Count number of students per course

```js
db.students.aggregate([
   { $group: { _id: "$course", total_students: { $sum: 1 } } }
])
```

Output:

```json
{ "_id": "BCA", "total_students": 5 }
{ "_id": "B.Tech", "total_students": 3 }
{ "_id": "B.Sc", "total_students": 2 }
```

🔹 Comparison: MapReduce vs Aggregation Pipeline

Feature	MapReduce	Aggregation Pipeline
Language	JavaScript functions	Built-in MongoDB operators
Speed	Slower	Faster
Use Case	Complex data transformations	Simple aggregations & summaries
Output	New collection	Immediate result set

🔹 Conclusion

> Both **Map-Reduce** and **Aggregation Pipeline** are powerful tools in MongoDB for performing data analysis.
> However, **Aggregation Pipeline is preferred** due to its **speed, simplicity, and built-in optimization**.

33) Draw and explain architecture of Distributed database system. State the reasons for building distributed database systems.

A distributed database system consists of loosely coupled sites (computer) that share no physical components and each site is associated a database system.

The software that maintains and manages the working of distributed databases is called distributed database management system.

The database system that runs on each site is independent of each other. Refer Fig. 6.1.1.

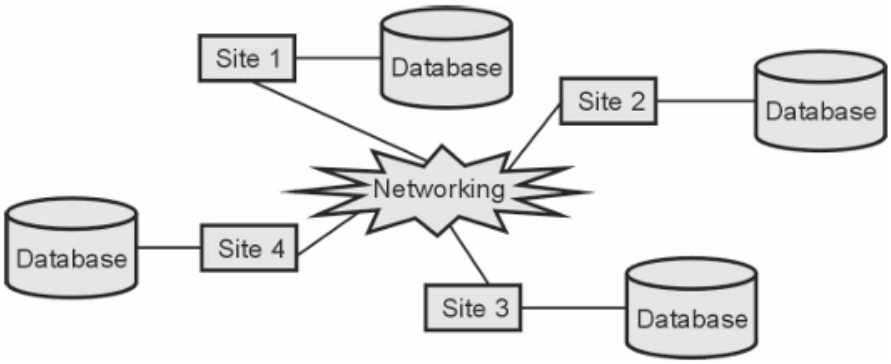


Fig. 6.1.1 Distributed database systems

The transactions can access data at one or more sites.

Advantages of Distributed Database System

1) There is **fast data processing** as several sites participate in request processing.
2) **Reliability and availability** of this system is high.
3) It possess **reduced operating cost.**
4) It is **easier to expand** the system by adding more sites.
5) It has improved **sharing ability and local autonomy.**

Disadvantages of Distributed Database System

1) The system becomes **complex** to manage and control.
2) The **security issues** must be carefully managed.
3) The system require **deadlock handling** during the transaction processing otherwise the entire system may be in inconsistent state.
4) There is need of some **standardization** for processing of distributed database system.

Difference between Centralized DBMS and Distributed DBMS

Distributed DBMS	Centralized DBMS
The database files are stored at geographically different locations across the network.	The database is stored at centralized location.
As data is distributed over the network , it requires time to synchronize data and thus difficult to maintain.	A centralized database is easier to maintain and keep updated since all the data are stored in a single location.
If one database fails, user can have access to other database files.	If the centralized database fails, then there is no access to a database.
It can have data replication as database is distributed. Hence there can be some data inconsistency.	It have single database system, hence there is no data replication. Therefore there is no data inconsistency.

Uses of distributed system :

1) Often distributed databases are used by organizations that have numerous offices in different geographical locations. Typically an individual branch is interacting primarily with the data that pertain to its own operations, with a much less frequent need for general company data. In such a situation, distributed systems are useful.
2) Using distributed system, one can give permissions to single sections of the overall database, for better internal and external protection.
3) If we need to add a new location to a business, it is simple to create an additional node within the database, making distribution highly scalable.

UNIT 6

34) Write short note on : i) Active databases ii) Deductive databases

i) Active Databases

Definition

Active databases are databases that can **automatically respond** to certain events or conditions **without user intervention**.
They contain built-in mechanisms to detect events and trigger actions.

Key Concept: ECA Rules

Active databases work using **ECA rules** (Event–Condition–Action):

Component	Meaning
Event (E)	Something that happens (INSERT, UPDATE, DELETE, time-based event)
Condition (C)	Checked before action. If true, action executes
Action (A)	Operation performed automatically (update, delete, send alert, etc.)

Example (Trigger Example)

"Whenever a new order is inserted, update the stock."

sql Copy code

```sql
CREATE TRIGGER update_stock
AFTER INSERT ON orders
FOR EACH ROW
BEGIN
   UPDATE product SET stock = stock - NEW.qty
   WHERE product_id = NEW.product_id;
END;
```

Uses

- Automatic auditing
- Maintaining integrity constraints
- Monitoring and alerting
- Automatic backup
- Fraud detection

Advantages

- Reduces manual checking
- Ensures consistency automatically
- Immediate response to critical events

ii) Deductive Databases

Definition

A deductive database is a database system that combines **traditional data storage** with **logical reasoning**. It uses **facts + rules** to derive **new, logical information** automatically.

Components

Component	Description
Facts	Base data stored in tables
Rules	Logical statements written in languages like Prolog or Datalog
Queries	Questions that may require logical inference

✅ Simple Example (Layman level)

Suppose you store only this data:

- "Ravi is the father of Amit"
- "Amit is the father of Sohan"

These are **facts**.

Now you ask:

"Is Ravi the grandfather of Sohan?"

Even though we **never stored** that fact,
the deductive database can **figure it out automatically** using a rule:

Rule:

"If A is parent of B, and B is parent of C → A is grandparent of C."

Using this rule, the database **deduces**:
➡️ Ravi → Amit → Sohan
✅ Ravi is grandfather of Sohan

Example

Facts:

```scss
parent(john, david)
parent(david, sam)
```

Rule:

```scss
grandparent(X, Z) :- parent(X, Y), parent(Y, Z).
```

Query:

```scss
grandparent(john, sam)?
```

Answer: TRUE (derived from rule → not stored directly)

Features

- Supports recursive queries
- Logical inference capability
- Works with Datalog, Prolog-like rules

Uses

- AI and expert systems
- Knowledge-based systems
- Semantic web
- Network analysis
- Natural language processing

Advantages

- Allows deriving new knowledge
- Handles complex relationships easily
- Supports intelligent querying

Features of Deductive Databases

1. Support for Facts and Rules

Stores facts (data) and rules (logic) in the same database.

2. Logical Inference

Database engine can derive new information automatically using rules.

3. Declarative Query Language

Uses logic-based languages like Datalog, where you specify what you want, not *how* to compute it.

4. Recursion Support

Rules can be recursive.
Example: finding all ancestors or pathfinding in a graph.

5. High Expressiveness

Can express complex queries that are difficult in SQL.

6. Knowledge Representation

Useful for AI, expert systems, semantic web, etc.

Advantages of Deductive Databases over Traditional Databases

Traditional Database	Deductive Database
Stores data only	Stores **data + rules**
Limited to SQL queries	Supports **logical reasoning**
Cannot derive new facts	Can **automatically infer** new facts
No support for recursion in standard SQL	Natural support for **recursive rules**
Complex queries require hard programming	Simple rule-based definitions
Less suitable for AI	Designed for **AI and knowledge-based systems**

Conclusion

- **Active databases** automatically respond to events using **ECA rules (Triggers)**.
- **Deductive databases** use **logic rules** to infer new information from stored facts.

35) What is the significance of XML databases? Explain with proper example when to use XML database.

Significance of XML Databases

An **XML database** is a database designed to **store, query, and manage XML documents** efficiently.
XML (eXtensible Markup Language) is used to represent **hierarchical, semi-structured data**.

XML databases are useful when data:

- is **not strictly tabular**
- has **nested structure**
- needs to be **shared across different platforms and applications**
- requires **flexible schema**

XML databases support **XQuery**, **XPath**, and DOM operations for fast searching in XML documents.

Why XML Databases Are Significant

1. Handles Semi-Structured Data

Relational databases require a fixed schema (rows–columns).
XML databases can store **irregular and hierarchical data**.

Example:

xml Copy code

```xml
<Student>
  <Name>Karan</Name>
  <Address>
    <City>Pune</City>
    <State>Maharashtra</State>
  </Address>
</Student>
```

Different students may have different structures. XML DBs handle this easily.

2. Platform and Application Independent

XML is universally readable.

Many systems (*Java, .NET, web applications*) use XML for data exchange.

XML databases make sharing easier.

---

3. Self-Describing Data

XML tags carry meaning. Relational tables need separate schema definitions.

Example:

```xml
<Price currency="INR">500</Price>
```

Contains value + metadata.

4. Ideal for Web Services & APIs

SOAP, RSS feeds, configuration files, and many APIs use XML.

XML databases store and retrieve such documents efficiently.

---

5. Support for Hierarchical Queries

XML DBs allow XPath/XQuery to directly search nested data.

Example XQuery:

```xquery
for $x in /Library/Book[Author="James Clear"]
return $x/Title
```

6. Better for Documents Than RDBMS

XML databases are optimized for:

- large documents
- technical manuals
- legal documents
- e-commerce catalogs
- configurations

RDBMS is not ideal for these formats.

When to Use XML Databases? (With Examples)

✅ 1. When data is hierarchical

Example:

Employee → Departments → Projects

XML naturally represents tree structures.

✅ 2. When schema changes frequently

Example:

Product details in e-commerce:

Some products have "color", some have "size", some have "warranty".

XML can store flexible attributes without altering database schema.

✅ 3. When exchanging data across applications

Example:

A banking system sends customer details to an insurance company in XML.

XML database can store and validate such structured documents.

✅ 4. When storing documents

- Legal contracts
- Research papers
- Technical documentation
- Medical records

These are naturally represented as XML.

✅ 5. For Web Data & Configuration Files

Web services use XML extensively:

- SOAP
- XHTML
- SVG
- Android layouts

  XML DBs fit naturally.

✅ 6. When querying complex nested data

Example: Find all books where publisher is inside nested nodes:

```xml
<Book>
  <Details>
     <Publisher>McGraw Hill</Publisher>
  </Details>
</Book>
```

Using XPath:

```xquery
//Publisher[text()="McGraw Hill"]
```

Conclusion

XML databases are invaluable when:

- Data is **hierarchical or semi-structured**
- Frequent schema changes occur
- Systems must **exchange data across platforms**
- Applications store or query **document-based data**

RDBMS works best for structured data, while XML databases handle flexible, nested, document-oriented data efficiently.

36) Difference between relational databases and object relational databases with example

✅ Difference Between Relational Databases and Object–Relational Databases

Feature	Relational Databases (RDBMS)	Object–Relational Databases (ORDBMS)
Data model	Pure **tables** (rows & columns)	Supports **tables** + **objects** (classes, inheritance, methods)
Type of data handled	**Structured data only**	Can handle **structured** + **complex data** (images, videos, custom objects)
Data Types	Only simple types (INT, CHAR, DATE, etc.)	Allows **user-defined types (UDT)**, multimedia types
Relationships	Only relational constraints (PK, FK)	Supports **object relationships** + relational constraints
Schema Flexibility	Fixed schema	More flexible — supports object attributes
Query Language	SQL	Extended SQL (Object SQL / OSQL)
Object Support	No concepts like inheritance, encapsulation	Supports **inheritance**, **methods**, **encapsulation**
Ideal Use Case	Banking, payroll, inventory (simple structured data)	CAD systems, multimedia DB, GIS, complex applications
Performance	Better for pure tabular operations	Better for **complex/large objects**, but slightly heavier
Storage	Stores only scalar values	Stores **objects**, large objects, collections

✅ Simple Example

Relational Database Example (RDBMS)

A **Student** table:

RollNo	Name	Course
1	Karan	CSE

Object–Relational Database Example (ORDBMS)

You can store **objects** directly, e.g., an Address object:

```pgsql
TYPE AddressType AS OBJECT (
   house_no NUMBER,
   city VARCHAR2(20),
   pin NUMBER
);
```

Then use it in a table:

```pgsql
CREATE TABLE Student (
  RollNo NUMBER,
  Name VARCHAR2(20),
  Address AddressType
);
```

This allows:

```sql
SELECT S.Address.city FROM Student S;
```

So ORDBMS supports **custom types, methods, and object-like behavior**, which is not possible in a pure RDBMS.

✅ Summary in One Sentence

- **RDBMS** = only tables + simple data
- **ORDBMS** = tables + objects, inheritance, user-defined types

37) Describe the significance of JSON data type and object. Discuss with syntax all JSON data types with suitable example.

✅ Significance of JSON Data Type and JSON Object

1. JSON (JavaScript Object Notation)

JSON is a lightweight, text-based format used to store and exchange data.

It is widely used in web applications, APIs, mobile apps, NoSQL databases, etc.

⭐ Significance of JSON

1. Human-readable & easy to understand

JSON looks like simple key–value pairs, so both humans and machines can read it easily.

2. Language independent

JSON can be used in Java, Python, C#, PHP, JavaScript, etc.

3. Ideal for APIs & Web services

Most REST APIs return data in JSON format.

4. Supports semi-structured data

Unlike relational tables, JSON can store:

- nested objects
- arrays
- dynamic fields

This makes JSON perfect for flexible data.

5. Used in NoSQL databases

Databases like MongoDB, CouchDB, and Firebase store records as JSON documents.

🧩 What is a JSON Object?

A JSON object is a collection of key–value pairs, written inside curly braces `{}`.

Example

```json
{
  "name": "Karan",
  "age": 22,
  "skills": ["Java", "Python"]
}
```

📌 JSON Syntax Rules

- Data is in key : value format
- Keys must be strings in double quotes
- Values can be:
  - String
  - Number
  - Boolean
  - Null
  - Object
  - Array

⭐ All JSON Data Types with Syntax + Examples

Below are all JSON data types explained clearly.

---

1) String

Syntax

```json
"key": "value"
```

Example

```json
"name": "Karan"
```

2) Number

No quotes required.

Example

```json
"age": 22,
"marks": 85.5
```

---

3) Boolean

Either `true` or `false`.

Example

```json
"isStudent": true
```

4) Null

Represents empty value.

Example

```json
"middleName": null
```

---

5) Object

A set of key–value pairs inside `{}`.

Example

```json
"address": {
  "city": "Pune",
  "pincode": 411001
}
```

6) Array

List of values inside `[]`.

Example

```
"skills": ["C", "C++", "Java"]
```

Array can contain numbers, objects, or other arrays.

⭐ Complete JSON Example Using All Types

```
{
  "name": "Karan",
  "age": 22,
  "isStudent": true,
  "middleName": null,
  "skills": ["Java", "Python", "SQL"],
  "address": {
    "city": "Pune",
    "pin": 411001
  }
}
```

🎯 When JSON is Used?

- REST API communication
- Mobile apps (Android/iOS)
- NoSQL databases (MongoDB documents)
- Configurations
- IoT data formats

📝 Conclusion

JSON is a universal data format used for storing and exchanging semi-structured, flexible, human-readable data.

It supports multiple data types and is widely used in modern web and database systems.

38) Write a short note on Main Memory Databases

Short Note on Main Memory Databases (MMDBs)

Definition

A **Main Memory Database (MMDB)** is a database system where the **entire data is stored in main memory (RAM)** instead of disk storage.
Because RAM access is extremely fast, MMDBs provide **high-speed transaction processing** and **real-time performance**.

Key Features

1. Very High Speed

- RAM access is much faster than disk access.
- Queries, transactions, and index lookups execute **10–100 times faster**.

2. Real-Time Processing

Used in applications where **immediate response** is essential, such as:

- Stock trading
- Telecom switching
- Online gaming
- Real-time analytics

---

3. Simple Storage Structure

Since data is stored in RAM:

- No disk seek time
- Optimized data structures like T-trees and hash tables are used

4. Durability with Backup

Although MMDBs store data in RAM, they ensure durability using:

- Periodic checkpointing
- Log files stored on disk
- Replication (copy on another server)

This keeps data safe even after power failure.

---

5. Reduced Overhead

No need for buffer management, caching, disk I/O scheduling, etc., so DBMS becomes simpler and faster.

Advantages

- Extremely fast performance
- Ideal for read-heavy and real-time systems
- Minimal latency
- Efficient index operations

Disadvantages

- RAM is costlier than disk
- Risk of data loss if durability mechanisms fail
- Limited storage capacity

Examples

- Redis (in-memory key-value store)
- SAP HANA
- Memcached
- VoltDB

Applications

- Banking transactions
- Leaderboards in games
- Caching systems
- Sensor data processing
- Stock exchange systems

Conclusion

Main Memory Databases provide **ultra-fast, real-time data access** by storing all data in RAM.
They are essential in environments where processing speed matters more than storage cost.

39) What is object relational database system. Explain Table inheritance with example

⭐ Object–Relational Database System (ORDBMS)

An **Object–Relational Database System** combines features of:

1. **Relational Databases (RDBMS)** → tables, rows, columns, SQL
2. **Object-Oriented Databases (OODBMS)** → objects, classes, inheritance, methods

It extends the relational model to support **complex data types** such as:

- User-defined types (UDTs)
- Nested objects
- Arrays and lists
- Multimedia data
- Table inheritance

Why ORDBMS?

Traditional relational databases handle only simple data (INT, TEXT).
Modern applications need to store:

- images, videos, locations
- complex objects (Customer → Address → Orders)
- hierarchical relationships

ORDBMS supports this without losing SQL.

Examples

- PostgreSQL
- Oracle Object-Relational Features
- IBM DB2
- Informix

⭐ Table Inheritance in ORDBMS

Table Inheritance allows one table to *inherit* the structure (columns) of another table, just like class inheritance in OOP.

It is similar to saying:

👉 **Child Table** = Parent Table + Extra Columns

This helps represent **hierarchical** or **"is-a" relationships**.

✅ Simple Example (Easy to Understand)

Parent Table: Vehicle

sql Copy code

```sql
CREATE TABLE Vehicle (
  id INT PRIMARY KEY,
  brand TEXT,
  model TEXT
);
```

Child Table: Car (inherits Vehicle)

A Car *is a* Vehicle + has some extra properties.

sql Copy code

```sql
CREATE TABLE Car (
  seats INT,
  fuel_type TEXT
) INHERITS (Vehicle);
```

Child Table: Bike (inherits Vehicle)

sql Copy code

```sql
CREATE TABLE Bike (
  engine_cc INT
) INHERITS (Vehicle);
```

✅ How Table Inheritance Works

1. Child table automatically gets parent columns

Car table now has:

- id
- brand
- model
- seats
- fuel_type

Bike table now has:

- id
- brand
- model
- engine_cc

2. Querying Parent Table Returns All Child Rows

```sql
SELECT * FROM Vehicle;
```

This returns:

- all Vehicles
- all Cars
- all Bikes

This is useful for applications like transport management or employee hierarchies.

⭐ Advantages of Table Inheritance

- Represents hierarchical data naturally
- Reduces redundancy (common columns stored once)
- Allows polymorphism (query parent to get all children)
- Maintains relational structure + object behavior

⭐ Conclusion

Object–Relational Database Systems extend RDBMS to support object features.
Table inheritance allows tables to behave like classes, supporting **reusability, hierarchy, and complex data modeling**, especially in modern applications.

40) Write a short note on complex data types : i) Semi-structured data ii) Features of semi-structured data models

i) Semi-structured Data (Short Note)

Semi-structured data is a type of data that **does not follow a strict fixed schema** like relational databases, but it still contains **tags, markers, or structural elements** that organize the data.
It lies **between structured and unstructured data.**

Key Points

- No fixed schema → schema can change from record to record.
- Uses **self-describing formats** like XML, JSON, HTML.
- Ideal for storing **heterogeneous, flexible, evolving data.**
- Common in web applications, APIs, NoSQL databases.

Example

```json
{
  "name": "Karan",
  "skills": ["React", "Node", "MongoDB"],
  "experience": { "years": 2, "domain": "Full Stack" }
}
```

Each record may have different fields → structure is flexible.

ii) Features of Semi-structured Data Models (Short Note)

Feature	Explanation
Flexible Schema	No predefined schema; users can add/remove fields anytime.
Self-describing Structure	Data carries its own structure using tags (XML) or keys (JSON).
Hierarchical or Graph-based	Supports nested objects and hierarchical relationships easily.
Supports Irregular Data	Works well when data items have missing values or different attributes.
Easily Extensible	New fields or nodes can be added without changing old data.
Human & Machine Readable	JSON, XML can be understood easily by both humans and programs.
Suitable for Web & NoSQL Storage	Used in MongoDB, Cassandra, CouchDB, APIs, configuration files.

41) Describe spatial data like Geographic data and Geometric data.

Spatial Data

Spatial data refers to any data that represents **objects or features located in space.**
It describes **where things are** and often includes shape, size, and location.

Spatial data is broadly divided into:

1. **Geographic Data (Geo-spatial data)**
2. **Geometric Data (Spatial geometry data)**

1. Geographic Data

Geographic data represents **real-world locations on Earth's surface.**
It includes coordinates based on **latitude & longitude.**

Key Points

- Tied to the Earth's geography.
- Uses coordinate systems: **GPS, WGS84, Latitude-Longitude.**
- Used in **Maps, GIS systems, Google Maps, Navigation Apps.**

Examples

Feature	Example
City location	Pune = (18.5204° N, 73.8567° E)
Rivers, mountains	Location of Himalayas
Roads & routes	Google Maps route data
Countries, states	Boundaries on a map

2. Geometric Data

Geometric data represents **shapes and objects in a mathematical space** (2D or 3D), not necessarily tied to Earth.

Key Points

- Represents geometric shapes: **points, lines, polygons.**
- Used in **computer graphics, CAD, gaming, simulations, vector graphics.**
- Does **not require** latitude-longitude.
- Uses **Cartesian coordinates (x, y, z).**

Examples

Shape	Example
Point	(4, 7) in a 2D plane
Line	Line from (2,3) to (7,9)
Polygon	Triangle, rectangle coordinates
3D Object	Cube represented with (x, y, z) vertices

Difference Summary

Geographic Data	Geometric Data
Based on Earth's surface	Based on math coordinates (x, y, z)
Uses latitude & longitude (GPS)	Uses Cartesian geometry
Used in GIS, mapping, navigation	Used in graphics, CAD, modelling
Represents natural features	Represents artificial shapes

42) Explain how encoding and decoding of JSON object is done JAVA with example.

⭐ 1. JSON Encoding (Java object → JSON string)

Encoding means **converting Java data** (objects, maps, lists) **into JSON format**.

Example Using Gson

```java
import com.google.gson.Gson;

class Student {
    String name;
    int age;
    String city;
}

public class EncodeJSON {
    public static void main(String[] args) {
        Gson gson = new Gson();

        Student s = new Student();
        s.name = "Rohit";
        s.age = 21;
        s.city = "Pune";

        String jsonString = gson.toJson(s);   // Encoding
        System.out.println(jsonString);
    }
}
```

Output

```json
{"name":"Rohit","age":21,"city":"Pune"}
```

This process is called **JSON encoding or serialization**.

⭐ 2. JSON Decoding (JSON string → Java object)

Decoding means **converting JSON data back into a Java object**.

Example Using Gson

```java
import com.google.gson.Gson;

class Student {
    String name;
    int age;
    String city;
}

public class DecodeJSON {
    public static void main(String[] args) {
        Gson gson = new Gson();

        String jsonString = "{\"name\":\"Rohit\",\"age\":21,\"city\":\"Pune\"}";

        Student s = gson.fromJson(jsonString, Student.class);   // Decoding

        System.out.println("Name: " + s.name);
        System.out.println("Age: " + s.age);
        System.out.println("City: " + s.city);
    }
}
```

Output

```makefile
Name: Rohit
Age: 21
City: Pune
```

This process is called **JSON decoding or deserialization**.

⭐ Summary Table (Easy to Write in Exam)

Operation	Meaning	Java (Gson) Function
Encoding	Java object → JSON string	`toJson(object)`
Decoding	JSON string → Java object	`fromJson(jsonString, Class)`

⭐ Short Exam Answer

JSON encoding converts a Java object into a JSON string using `toJson()`.
JSON decoding converts a JSON string back into a Java object using `fromJson()`.

All the Best!!

- **Karan Salunkhe ☺**